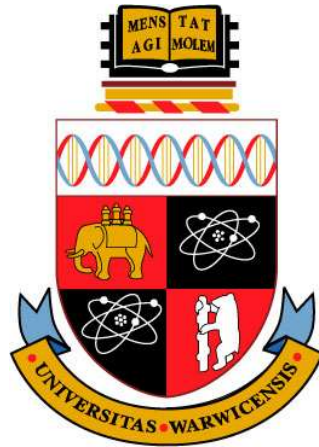




CS351 Final Report: End-to-end evaluation of
pre-processing placement in embedded FPGA CNN
pipelines using an Ultra96-V2 and AMD DPU IP

Author: Mateusz Gremlik - 2261844

Supervisor: Eduardo Wächter

University of Warwick

April 2026

1 Abstract

This project investigates how the placement of image pre-processing affects end-to-end performance in embedded FPGA CNN pipelines when inference has already been accelerated by AMD’s Deep Learning Processing Unit (DPU). The work targets the Ultra96-V2, a Zynq UltraScale+ MPSoC platform that supports heterogeneous execution across an Arm-based processing system and FPGA programmable logic. The main question is whether moving pre-processing from software into programmable logic still produces meaningful gains once inference itself is no longer the dominant.

To answer this question, an embedded image-classification pipeline was designed and evaluated with three workloads with different front-end characteristics: Dogs vs. Cats using MobileNetV2, MNIST using a custom CNN, and American Sign Language recognition using ResNet50. Across these three workloads, the DPU configuration was kept constant while alternative pre-processing implementations were compared, including CPU processing, a naive programmable-logic kernel, and an improved kernel, both developed in High-Level Synthesis. A custom INA226-based power-measurement board and automated benchmarking framework were also used to collect latency, throughput, power, and energy data under controlled conditions.

The results show that pre-processing can remain a substantial contributor to total runtime and energy even after inference has been offloaded to the DPU. For workloads with significant and regular front-end processing, programmable-logic pre-processing improved throughput and reduced energy use. In the strongest cases a speed-up of $2.31\times$ was achieved over baseline. For smaller or simpler inputs, the benefit was limited by invocation and transfer overheads. These findings show that efficient embedded CNN deployment depends on whole pipeline partitioning. This project further demonstrates that HLS provides a viable way to add model specific front end acceleration while preserving a reusable DPU-based flow.

Keywords: *Edge AI; FPGA acceleration; Ultra96-V2; Vitis AI; Deep Learning Processing Unit (DPU); CNN inference; pre-processing offload; energy efficiency*

Contents

1	Abstract	2
2	Introduction	5
3	Objectives	6
4	Background	7
4.1	CNN	7
4.2	Datasets	7
4.3	FPGAs	8
4.3.1	MPSoC Architecture	8
4.3.2	PetaLinux	9
4.4	DPU	10
4.4.1	Internal Architecture	11
4.4.2	The DPU Workflow	11
4.5	Hardware-Software Co-Design with Vitis AI	12
4.5.1	AI Optimizer and Quantizer	12
4.5.2	AI Compiler and Vitis AI Runtime (VART)	13
5	Literature Review	14
5.1	Highly Specialised FPGA Accelerators	14
5.2	Reusable Overlays	16
5.3	End-to-end Studies	17
5.4	Conclusions	19
6	System Design	20
6.1	System Architecture	20
6.2	Execution Flow	20
7	Implementation	22
7.1	Development Environment	22
7.1.1	Windows Subsystem for Linux (WSL)	22
7.1.2	Docker and Vitis AI	22
7.1.3	Ultra96v2 Board Support Packages (BSPs)	22
7.2	Hardware Platform	23
7.2.1	DPU Integration	23
7.2.2	Pre-Processing Kernel	24
7.3	Software Platform	27
7.3.1	MobileNetV2 Dogs vs. Cats Model	27
7.3.2	MNIST Model	27
7.3.3	ResNet50 ASL Model	28
7.3.4	Software Application	28
7.3.5	PetaLinux Platform	29
8	Experimental Methodology	30
8.1	Performance Metrics	30

8.2	Test Platform	32
8.2.1	GPU/CPU	32
8.2.2	Ultra96-V2	33
8.2.3	Ultra96-V2 Power Measurement Sub-Board	34
8.3	Validation	35
8.4	Testing Methodology	36
9	Results	38
9.1	Dogs vs. Cats with MobileNetV2	38
9.1.1	No-Resize Results	38
9.1.2	Resize Results	40
9.1.3	Discussion	41
9.2	MNIST with a Custom CNN	42
9.2.1	End-to-End Results	43
9.2.2	Discussion	43
9.3	ASL with ResNet50	44
9.3.1	CPU and GPU Results	44
9.3.2	FPGA End-to-End Results	45
9.3.3	Live Image Processing	46
9.3.4	Discussion	47
9.4	Overall Discussion	48
10	Conclusions	50
10.1	Objective-to-Evidence Summary	51
11	Further Work	51
12	Bibliography	53
A	Appendix	57
A.1	Author's (Informal) Assessment of the Project	57
A.2	GitHub Repository with Project Files	58

2 Introduction

Edge Artificial Intelligence (Edge AI) forms part of a wider move towards deploying machine-learning systems close to the point of data generation, on embedded or local computing platforms, rather than exclusively in remote infrastructure [1] [2] [3]. Within this context, image classification with Convolutional Neural Networks (CNNs) provides a useful embedded workload, since it combines substantial computational demand with a continuous requirement to acquire, transform, and move visual data through the system. For embedded platforms, the behaviour of such pipelines is shaped not only by the neural network itself, but also by the surrounding stages that prepare input data for inference.

FPGA-based platforms are well suited to this setting because they combine programmable hardware acceleration with the flexibility of a general-purpose processing system [4] [5]. On Zynq UltraScale+ MPSoC devices, this creates a heterogeneous environment in which software and hardware can be partitioned across the processing system and programmable logic. AMD’s Vitis AI flow extends this model further through the Deep Learning Processing Unit (DPU), which provides a reusable inference engine for quantised CNN deployment [6]. Once inference is mapped to such an accelerator, however, end-to-end performance depends on more than the execution of the CNN alone. Operations such as resizing, colour conversion, normalisation, and tensor preparation remain necessary for every input and can account for a substantial share of total runtime and energy use [7] [8].

This project investigates that system-level problem through the design and evaluation of an FPGA-based image-classification pipeline on the Ultra96-V2 platform [9]. The study is centred on pre-processing placement around a fixed DPU-based inference stage, with the aim of determining how hardware–software partitioning affects end-to-end latency, throughput, and energy efficiency. To examine this, multiple workloads and pre-processing implementations are developed and compared on a common embedded platform. The result is a controlled evaluation of how far programmable logic can improve overall pipeline behaviour once CNN inference has already been accelerated.

3 Objectives

The objectives for this project are therefore:

1. Set up development environment including FPGA and APU boot files, Vitis AI workflow, and TensorFlow. [Must]
2. Reproduce a baseline Convolutional Neural Network (CNN) using the TensorFlow API for image classification. [Must]
3. Use AMD Vitis AI tools to quantise and compile the trained model for FPGA/DPU inference and deploy it on the Ultra96-V2 FPGA. [Must]
4. Verify the correctness and stability of inference results between floating-point (CPU/GPU) and quantised (FPGA) models to confirm functional equivalence. [Must]
5. Evaluate energy efficiency by recording throughput, latency, power, and energy-per-inference metrics. [Must]
6. Compare and analyse inference performance and energy efficiency between FPGA (DPU), GPU and CPU implementations. [Must]
7. Design and implement pre-processing pipeline in hardware to reduce APU load and reduce latency. [Should]
8. Perform model optimisation to test energy–accuracy trade-offs. [Should]
9. Automate deployment and testing workflow using scripts to ensure reproducibility. [Should]
10. Extend the data pipeline to include real-time image data processing. [Could]
11. Implement more complex image recognition CNN models to test boundary of FPGA capability. [Could]
12. Evaluate the usefulness of FPGA technology in edge-AI scenarios for CNN image recognition tasks. [Must]

4 Background

4.1 CNN

CNNs are a form of neural network that are especially suited for spatial data, such as images. CNNs apply small learnable kernels across an image to extract local features, building increasingly abstract representations across layers. This architecture was popularised early in digit recognition and later has become dominant in image classification [10] [11].

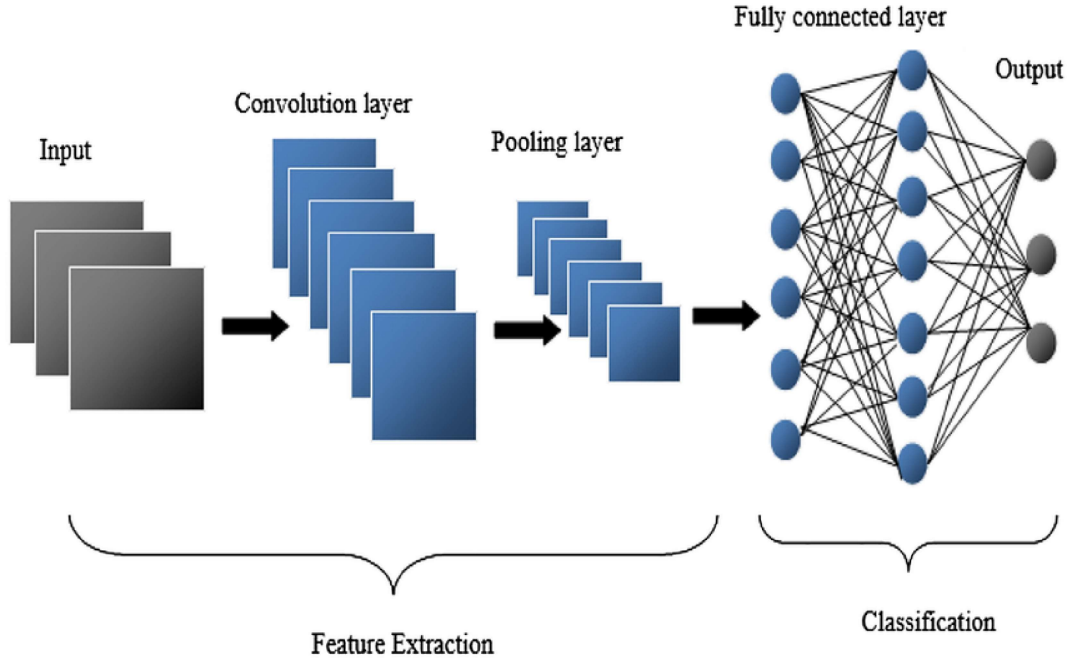


Figure 1: General Structure of CNN. Image from [12]

One of the core operations of a CNN, convolution layers, involves many millions of multiply-accumulate operations. These make up for the majority of processing done by CNNs. Due to the nature of convolution, it is highly repetitive and parallelisable making it a well suited for hardware acceleration [13]. However, achieving good performance requires considering memory access patterns and data movement beyond just arithmetic throughput. This is especially true in embedded systems with limited memory bandwidth [13] [5].

4.2 Datasets

Image-classification research commonly uses datasets with different levels of visual complexity. Natural-image datasets such as Dogs vs. Cats contain unconstrained

colour photographs with variation in pose, lighting, background, scale, and viewpoint, making them useful for evaluating binary classification under realistic conditions [14].

MNIST is a standard handwritten digit-recognition dataset consisting of 60,000 training images and 10,000 test images, each represented as a normalised 28×28 grayscale image of a decimal digit. Because of its small input size, limited class set, and highly standardised format, it is widely used as a compact benchmark for classifier development and validation [15].

Sign-language image datasets, including ASL benchmark sets, contain labelled hand signs and are commonly used in gesture recognition, sign-language understanding, and assistive human-computer interaction. Compared with simpler benchmarks, they place greater emphasis on fine-grained spatial differences in hand shape and finger position, making them a more demanding visual-classification problem [16].

4.3 FPGAs

A Field-Programmable Gate Array (FPGA) is a reconfigurable digital device composed of configurable logic and programmable interconnect, enabling hardware circuits to be built after manufacturing [4]. Unlike CPUs/GPUs which execute fixed instruction sets, FPGAs can be configured as custom data paths tailored to a workload. This makes them extremely attractive for edge systems where performance-per-watt and deterministic latency matter as there is less unused functionality than fixed-function processors [13] [4].

4.3.1 MPSoC Architecture

The Ultra96-V2 is based on a Zynq UltraScale+ MPSoC, meaning it combines a conventional processor subsystem with reconfigurable FPGA fabric in a single device [17]. The processing system (PS) hosts the Arm Application Processing Unit (APU), which runs Linux and is responsible for system-level tasks such as file I/O, networking, memory allocation, and controlling accelerator execution. The programmable logic (PL) is used to implement high-throughput hardware data paths that operate in a highly parallel and deterministic manner [17]. This separation of roles is fundamental to practical heterogeneous FPGA-based acceleration[18].

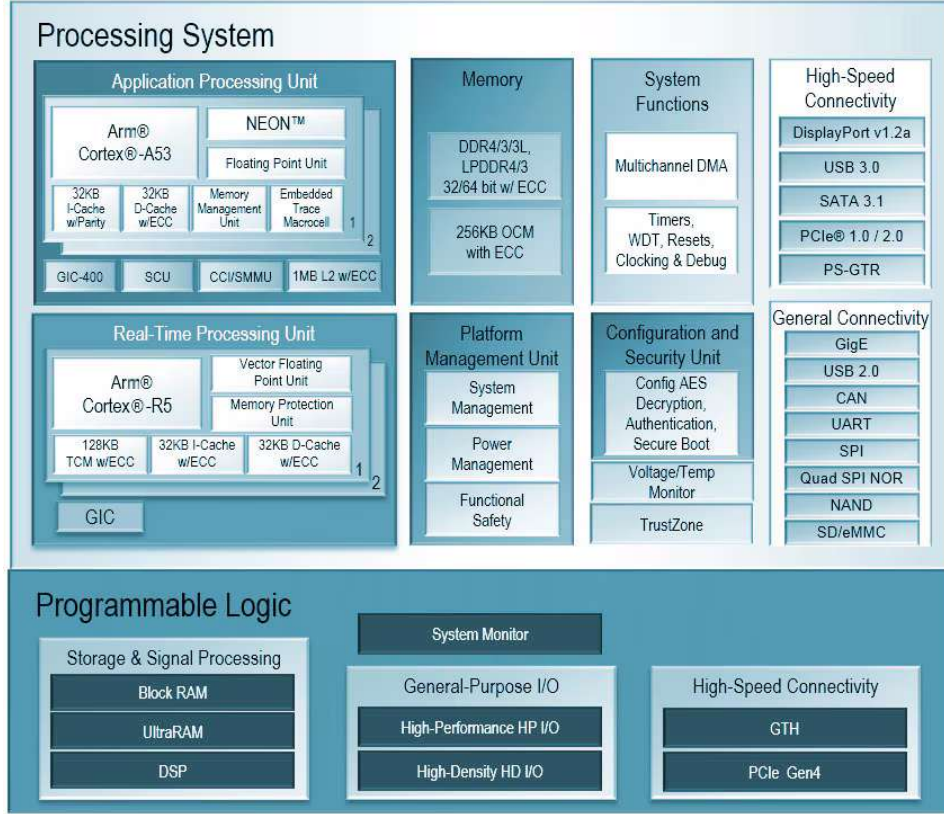


Figure 2: Diagram of Zynq Ultrascale+ Platform. Image from [19]

In an embedded image classification pipeline, the APU is typically responsible for acquiring images, handling application logic, and coordinating inference execution through the runtime libraries[20] [21]. The PL is then used for two main acceleration opportunities: CNN inference via the DPU, and application-specific hardware blocks such as pre-processing accelerators. Communication between the APU and PL is achieved through AXI interconnects, with data transferred using DMA engines into shared DDR memory [17]. This architecture enables a hardware–software co-design approach where latency-critical, high-throughput stages can be moved into PL while the APU maintains overall programmability[6] [21] [18].

4.3.2 PetaLinux

PetaLinux is AMD’s embedded Linux build environment for supported FPGA and MPSoC platforms. It is used to generate the bootable software platform associated with a hardware design, including the kernel, device tree, root filesystem, and board-specific configuration. In FPGA-based systems this coupling is important, since the operating system must reflect the implemented memory map, peripherals, clocks, and accelerator interfaces in order for runtime software and user-space applications

to access the deployed hardware correctly.

4.4 DPU

To overcome the challenges of traditional FPGA development, which often involves low-level hardware description languages (HDL) such as Verilog, SystemVerilog and VHDL, this project uses AMD’s DPU [6] [21]. The DPU is a programmable engine optimized for convolutional neural networks, implemented as a soft IP core within the FPGA’s PL. Unlike fixed-function hardware, the DPU is a micro-coded processor with its own specialized internal Instruction Set Architecture (ISA), allowing it to execute a wide variety of CNN layers without requiring the FPGA to be re-flashed for every new model[6] [21].

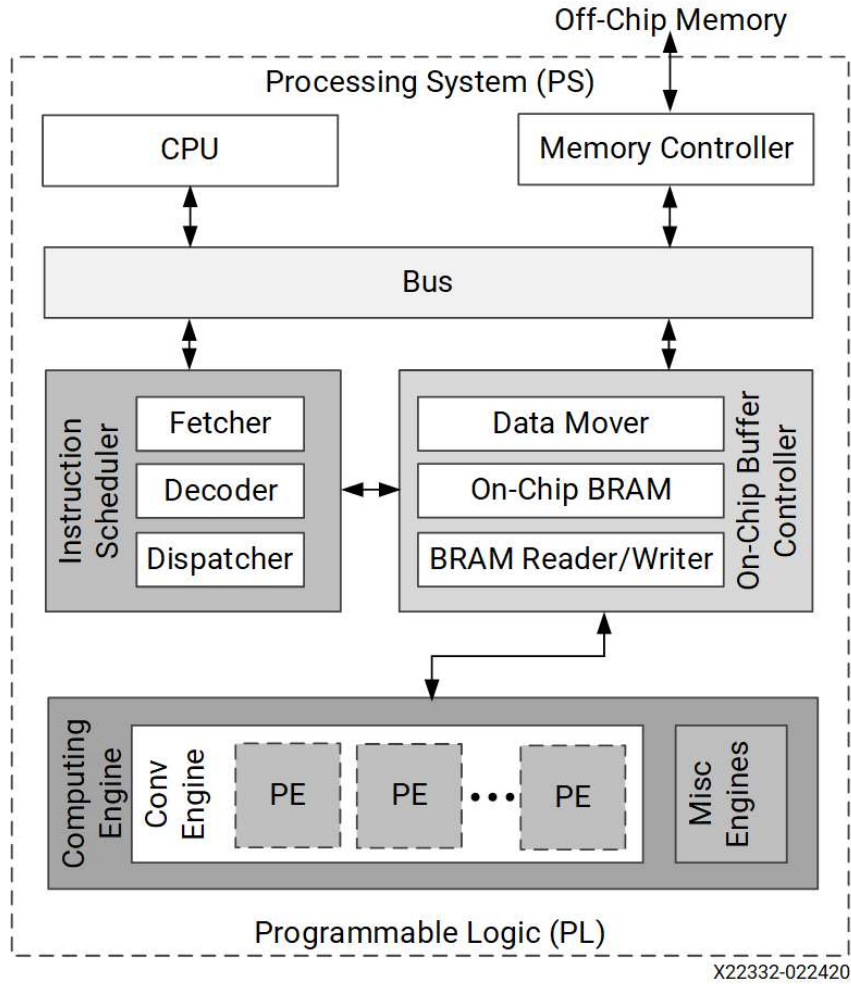


Figure 3: Ultra96-V2 DPU Hardware Architecture. Image from [21]

4.4.1 Internal Architecture

The DPU architecture is designed around a 'Matrix of Processing Engines' (PE) and a specialized data-flow controller. Its internal structure consists of several critical modules[6] [21]:

- **Instruction Fetch Unit:** This module interfaces with the Application Processing Unit (APU) via an AXI-Lite slave interface. It allows the software to set up the DPU's operating parameters, start tasks, and monitor status.
- **Buffer Controller Module:** This is the traffic controller of the DPU. It manages the high-speed AXI Master interfaces that fetch instructions from external DDR memory and move input/output feature maps and weights.
- **Computing Engine (PE Matrix):** Consists of a grid of processing elements that leverage the FPGA's Digital Signal Processing (DSP) slices to perform massively parallel Multiply-Accumulate operations. These PEs are optimized for high-throughput convolution.

Although the DPU is delivered as a 'soft' IP core in programmable logic, it is not a generic drop-in block. Successful instantiation depends on the underlying PL being able to support a dense, timing-critical DSP array with predictable interconnect and clocking. The DPU design assumes that there is sufficient contiguous DSP resources to form wide systolic PE matrices, high-capacity on-chip memory positioned close to the compute fabric, memory subsystem capable of sustaining multiple concurrent high-bandwidth AXI transactions, and clocking infrastructure that can support deeply pipelined data paths, in some configurations using separate clock domains for general logic and high-speed DSP operation[21].

For this reason, DPU configurations are typically targeted at specific platforms, where the architecture, DDR controllers, and PL resource distribution are designed to meet these constraints. Even if an FPGA can accommodate the required logic resources, inadequate DSP connectivity, memory bandwidth, or timing closure capability can prevent the DPU from operating[21].

4.4.2 The DPU Workflow

The DPU does not run a model in the traditional software sense. Instead, it executes a sequence of micro-coded instructions generated by the Vitis AI Compiler[22][21].

- **Instruction Fetch:** The DPU fetches its specialized binary instructions (.xmodel file) from a dedicated memory region in the DDR.

- **Scheduling and Decoding:** The internal scheduler decodes these instructions into low-level control signals for the PE matrix.
- **Parallel Execution:** The PE matrix processes the data in tiles, performing convolutions, pooling, and activations in a deep pipeline.
- **Synchronization:** Once a task is complete, the DPU generates an interrupt to the APU, signalling that the results are ready for the software layer to read.

4.5 Hardware-Software Co-Design with Vitis AI

Implementing an FPGA accelerator requires a bridge between high-level machine learning frameworks and low-level hardware. The Vitis AI development stack provides this through a multi-stage workflow.

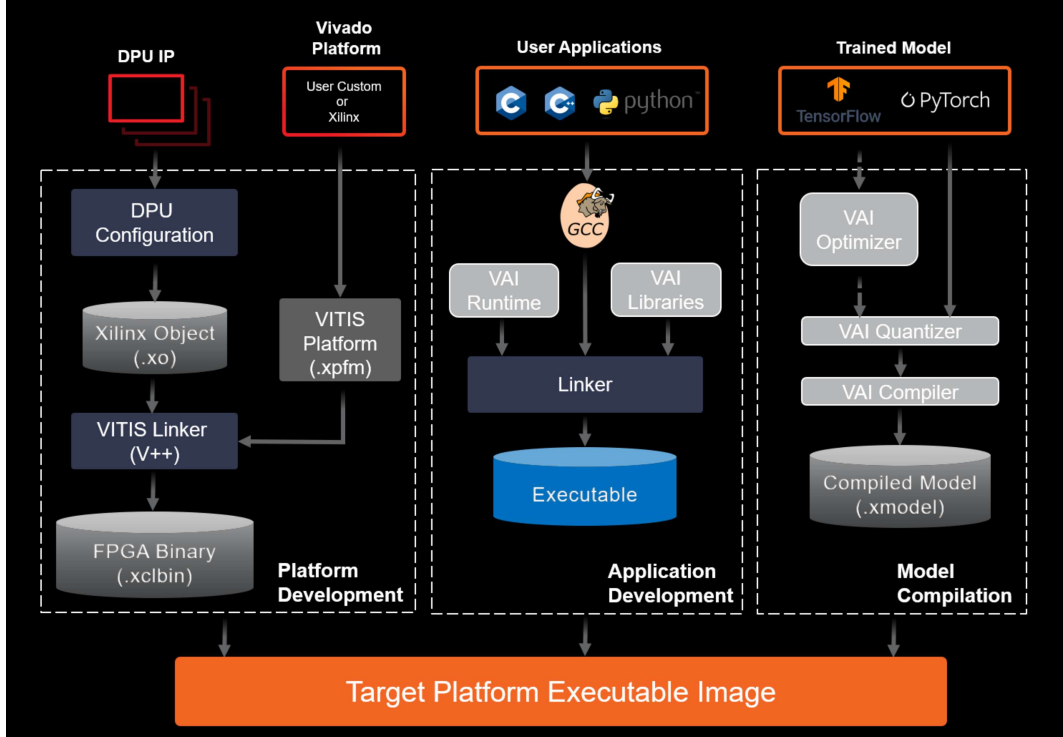


Figure 4: Vitis AI Workflow. Image from [22]

4.5.1 AI Optimizer and Quantizer

In software, neural networks typically use 32-bit floating-point weights. The AI Quantizer converts these into 8-bit integers. This process involves calibration, where the quantizer analyses the distribution of weights and activations in the model to minimize precision loss[22]. This 4x reduction in data width allows for 4x higher

memory bandwidth and significantly faster arithmetic in the FPGA’s DSP slices [23].

4.5.2 AI Compiler and Vitis AI Runtime (VART)

The AI Compiler takes the quantized model and maps it to the DPU’s specialized ISA. It analyses the model’s computation graph, optimizes the scheduling of operations to maximize PE utilization, and generates the final micro-code[21][22]. It also handles operator splitting, where layers not supported by the DPU are automatically routed back to the CPU for execution[22].

On the target board, the Vitis AI Runtime (VART) manages the interaction between the user’s Python or C++ application and the hardware. It handles buffer allocation, memory mapping, and the low-level system calls required to send data to and from the DPU [22]. This allows developers to treat the DPU as a standard library call, reducing development time.

5 Literature Review

Research on FPGA-based CNN deployment is largely focused on accelerating inference because it accounts for most of the arithmetic cost in many visual models. That avenue has produced a large body of work on neural accelerators, however, the wider hardware literature has shown that arithmetic throughput alone does not explain end-to-end performance. Sze et al. examine efficient DNN (Deep Neural Network) execution as a problem involving computation, data movement, and memory hierarchy, while Eyeriss demonstrates that reducing data movement is in itself an optimisation avenue because memory traffic can account for a large amount of energy consumption [24][25]. In embedded deployments, these concerns become more important because bandwidth, storage, and power budgets are much tighter than in datacentre settings [26].

5.1 Highly Specialised FPGA Accelerators

One distinct area of the literature addresses this problem through highly specialised FPGA inference engines generated for a particular model family and precision criteria. FINN is the best-known example. It targets binarised neural networks using a heterogeneous streaming architecture in which hardware resources are tailored per layer to meet a throughput goal. On a ZC706 embedded FPGA platform drawing less than 25 W, FINN reports up to 12.3 million classifications per second at 0.31 μ s latency on MNIST and 21,906 classifications per second at 283 μ s latency on CIFAR-10 and SVHN [27].

FINN-R extends its predecessor’s work from fully binarised networks to quantised neural networks with arbitrary precisions and adds analytical cost models for design-space exploration. Blott et al. report throughput up to 50 TOp/s on AWS F1 and 5 TOp/s on embedded devices, showing that the benefits of custom dataflow hardware are not limited to the binary case alone [28]. This turns reduced precision into a tunable variable [28]. At the same time, FINN-R reinforces the main trade-off already that is present in FINN - these gains are achieved by shaping the hardware around the model, which is fundamentally different from using a reusable inference overlay whose architecture remains fixed across workloads [28][27].

A broader set of more automated FPGA toolflows attempts the same objective of model-specific acceleration without adopting FINN’s specific architecture. fpgaConvNet presents CNN-to-FPGA mapping as primarily an optimisation problem. Likewise, DNNBuilder combines automatic design-space exploration with fine-grained pipelining and cache-aware memory organisation [29][30]. hls4ml pushes the same

idea into a more general open-source workflow, enabling model-to-hardware translation without requiring full custom RTL [31]. What these approaches all share in common is that they treat model itself as an object to be compiled into hardware [29][30][31]. They show how far performance can be pushed when the inference engine is allowed to vary, but they address a different question from a controlled evaluation that keeps inference fixed and varies only the surrounding data path [29][30][31].

TVM represents a different solution again, treating deployment primarily as a compiler problem rather than as the construction of one specific neural accelerator. Chen et al. present TVM as an end-to-end optimisation stack that exposes graph-level and operator-level optimisations, performs learning-based cost modelling, and targets a wide range of back ends including CPUs, GPUs, and FPGA-based accelerators [32]. Within, this context, its relevance to this work lies in performance portability rather than maximum specialisation [32]. A TVM-style workflow is well suited to considering how a DNN should be mapped across heterogeneous hardware in general [32].

Author(s), year	Type	Quantitative metrics
Umuroglu et al., 2017 [27]	Custom CNN accelerator	12.3 million classifications/s at 0.31 μ s on MNIST; 21,906 classifications/s at 283 μ s on CIFAR-10/SVHN; < 25 W.
Blott et al., 2018 [28]	Custom quantised CNN accelerator	Up to 50 TOP/s on AWS F1 and 5 TOP/s on embedded devices.
Venieris and Bouganis, 2019 [29]	Automated CNN-to-FPGA mapping	Up to 6.65 $\times$ GPU-matched performance and 2.94 $\times$ higher performance density than prior FPGA designs.
Zhang et al., 2018 [30]	Automated FPGA DNN builder	7.7 $\times$ latency reduction, 43 $\times$ BRAM reduction, up to 5.15 $\times$ higher performance, 4218 GOPS.
Fahim et al., 2021 [31]	ML-to-hardware workflow	Workflow paper; no single fixed benchmark.
Chen et al., 2018 [32]	Compiler optimisation stack	Framework paper; no single fixed benchmark.

Table 1: Highly specialised FPGA accelerators and related toolflows.

5.2 Reusable Overlays

In contrast to these model-specialised, vendor-supported inference overlays prioritise deployability and repeatability. AMD’s DPUCZDX8G is a representative of this design philosophy. According to its product guide, it is a configurable, microcoded computation engine for convolutional neural networks implemented in the programmable logic of Zynq UltraScale+ MPSoCs, with multiple configurations that scale resource use and parallelism [6]. The associated Vitis AI flow uses quantisation and compilation to generate the .xmodel to be executed by the DPU, while the quantizer’s inspector explicitly reports which operators are mapped to the DPU and which remain on the CPU [20]. Unsupported operators and surrounding application stages, such as pre-processing, can still dominate the total runtime even when the CNN core is accelerated [20][6].

The practicality of this overlay-based strategy is illustrated by Machura, Danilowicz and Kryjak, who compare a custom accelerator, FINN, and Vitis AI on the Ultra96-V2. Their study reports 196 fps for the custom accelerator, 111 fps for FINN, and 123.3 fps and 53.3 fps for two different Vitis AI implementations [33]. This comparison demonstrates a meaningful middle ground between specialisation and deployability on the same class of embedded MPSoC [33]. A reusable DPU overlay may not maximise throughput in every case, but it provides a stable and repeatable baseline [33][6][20].

Low-precision inference can provide a bridge between specialised dataflow accelerators and reusable vendor overlays. Jacob et al. show that integer-only inference can preserve accuracy close to floating-point inference while reducing storage requirements and enabling efficient execution on integer hardware [34]. Courbariaux et al. develop the same argument further with binarised neural networks, where weights and activations are constrained to ± 1 , drastically reducing memory size and replacing most arithmetic with bit-wise operations [35]. In deployment terms, once a model expects reduced-precision tensors, colour conversion, normalisation, scaling, and packing become part of the cost of inference [34][35]. Therefore, pre-processing placement is tied to the input format imposed by the accelerator interface [34].

The architecture of the CNN also affects whether front-end offload is likely to matter. ResNet established the viability of very deep residual networks, with the 152-layer model achieving 3.57% top-5 error on ImageNet while remaining easier to optimise than earlier very-deep models [36]. MobileNetV2, by contrast, was designed specifically for mobile and resource-constrained inference, using inverted residuals, linear bottlenecks, and depthwise separable convolutions to reduce computation and memory [37]. These model families stress different parts of an embedded pipeline

[36][37]. Heavier architectures can keep inference dominant, whereas lighter architectures can make front-end processing and data movement a much larger share of end-to-end time [36][37]. The value of hardware pre-processing is therefore fundamentally workload-dependent rather than universal [36][37].

Author(s), year	Type	Quantitative metrics
AMD, 2025 [6]	AMD DPU documentation	Architecture/configuration reference; not a benchmark study.
AMD, 2022 [20]	Vitis AI workflow documentation	Quantisation/compilation/runtime reference; not a benchmark study.
Machura et al., 2022 [33]	Ultra96-V2 custom vs FINN vs AMD DPU-based	196 fps custom, 111 fps FINN, 123.3 fps and 53.3 fps Vitis AI.
Jacob et al., 2018 [34]	Integer-only quantisation	Near-floating-point accuracy with about 4× lower storage than FP32.
Courbariaux et al., 2016 [35]	Binarised neural networks	Strong memory and arithmetic reduction; mainly accuracy-focused rather than FPGA benchmark-focused.
He et al., 2016 [36]	CNN architecture	ResNet-152 achieved 3.57% top-5 error on ImageNet.
Sandler et al., 2018 [37]	Lightweight CNN architecture	Mobile-focused accuracy/latency trade-off; no single headline figure needed here.

Table 2: Reusable overlays, quantisation, and model context.

5.3 End-to-end Studies

The literature most closely aligned with the main question presented is the smaller body of work that treats pre-processing and post-processing as primary acceleration targets. Li et al. argue explicitly that many image-classification accelerators optimise convolution but leave non-convolutional tasks such as loading, decoding, and scaling on general-purpose processors [7]. Their FPGA-based JPEG preprocessing accelerator addresses that void directly and reports 23.07 times higher energy efficiency than a CPU implementation, 4.21 times higher energy efficiency than a GPU implementation, and 2.52 times higher throughput than the CPU baseline for JPEG preprocessing [7]. In an end-to-end image-classification case study, they further report 44.03 times higher energy efficiency than the CPU implementation, 2.32 times higher energy efficiency than the GPU implementation, and 28.27 times higher throughput than the CPU solution [7]. These results show that front-end stages can remain important enough to justify dedicated hardware especially when inference is already accelerated [7].

Kang, Mathur, Veeramacheneni, Bailis, and Zaharia reach a closely related conclusion in the context of visual analytics on modern accelerators. They show that preprocessing, including decoding and resizing, can become the bottleneck in end-

to-end DNN execution, and they propose Smol, a system that jointly optimises preprocessing and inference by pipelining execution and placing preprocessing on the CPU or GPU in a hardware- and input-aware manner [38]. Across eight visual datasets, Smol achieves up to 5.9 times end-to-end throughput improvement at the same accuracy as visual analytics systems [38]. This presents preprocessing placement as a whole systems optimisation problem rather than just an implementation choice [38]. Even though the hardware differs from an FPGA SoC, the conclusions are relevant. Once inference becomes fast enough, preprocessing must be analysed together with it [38][7].

Cittadini, Marinoni, and Buttazzo provide perhaps the closest work. Their 2025 work presents a hardware accelerator designed specifically to support deep learning processor units by offloading the heavy stages before and after model inference. Tested with YOLO on an AMD Xilinx Kria KR260, the proposed design improves full-pipeline performance by a factor of 4.4, raising throughput from 13.6 fps to 60 fps [8]. It treats the DPU as a reusable inference core and complements it with custom hardware for deterministic surrounding tasks [8].

Author(s), year	Type	Quantitative metrics
Li et al., 2022 [7]	FPGA pre-processing accelerator	$23.07\times$ CPU energy efficiency, $4.21\times$ GPU energy efficiency, $2.52\times$ CPU throughput for preprocessing; $44.03\times$ CPU energy efficiency and $28.27\times$ CPU throughput end-to-end.
Kang et al., 2020 [38]	Joint preprocessing–inference optimisation	Up to $5.9\times$ end-to-end throughput improvement at fixed accuracy.
Cittadini et al., 2025 [8]	AMD DPU-based pre/post-processing support	$4.4\times$ full-pipeline speed-up; 13.6 fps to 60 fps.
Fabre et al., 2024 [26]	Embedded AI vision survey	Survey paper; no single benchmark. Uses metrics such as throughput, latency, bandwidth, and power efficiency.
Sze et al., 2017 [24]	DNN acceleration survey	Survey paper; no single benchmark. Focuses on compute, memory, data movement, and precision.
Chen et al., 2017 [25]	Energy-efficient CNN accelerator	35 fps and 278 mW for AlexNet convolution in one configuration.

Table 3: End-to-end studies and broader systems context.

5.4 Conclusions

Finally, the embedded-vision review by Fabre et al. expands the discussion of pre-processing placement by exploring embedded AI vision as a continuum from near-sensor to in-sensor processing [26]. Their review identifies metrics including sensor resolution, frame rate, bandwidth, throughput, latency, power efficiency, and scalability, and argues that the design trade-off is between flexibility and energy consumption [26]. Near-sensor systems process data close to its source using dedicated hardware or embedded components, while in-sensor systems push computation into the sensing components to reduce data movement and power [26]. This questions pre-processing placement within a broader discussion about where visual computation should occur [26].

Together, the literature suggests three main conclusions. Firstly, highly specialised FPGA accelerators can achieve exceptional throughput and efficiency, but they do so by tailoring hardware closely to the deployed model and reducing flexibility [27][28][29][30]. Secondly, reusable overlays such as AMD’s DPU are in a middle ground, sacrificing some peak performance in exchange for a far more practical deployment flow [6][20][33]. Finally, recent end-to-end studies show that once inference is accelerated, deterministic stages such as decoding, resizing, colour conversion, normalisation, and packing can become the remaining bottleneck [7][38][8]. What remains comparatively under-explored is a controlled evaluation of this issue in which the inference engine is held constant and only the placement and implementation of pre-processing are varied. This is the gap this study aims to address.

6 System Design

Across all workloads, execution follows the same high-level sequence: image loading, input preparation, inference, and output interpretation. The main architectural difference is in the placement of pre-processing, which can remain on the processing system or be offloaded to programmable logic.

6.1 System Architecture

The design combines an offline model-preparation flow with an on-board runtime application. Offline, each model is trained, quantised, compiled, and packaged for DPU execution. On the target, the runtime loads images, performs any CPU-side preparation, invokes the DPU through VART, and decodes the outputs.

At hardware level, the design follows the standard Zynq MPSoC split between the processing system and programmable logic. The processing system runs Linux and handles application control, memory management, and accelerator coordination. The programmable logic contains the DPU and, where required, a custom pre-processing kernel. Data is exchanged through AXI-connected shared DDR memory.

Figure 5 shows the partition used throughout the project. Control and general software execution remains in the processing system. The DPU and any custom pre-processing hardware are placed in programmable logic.

6.2 Execution Flow

Raw image data is loaded into memory and follows one of three pre-processing paths before inference. In the CPU path, preparation is executed entirely in software. In the programmable-logic paths, the model-specific front end is split between software and hardware, where the kernel produces data in the format required by the DPU. Output interpretation is then returned to software.

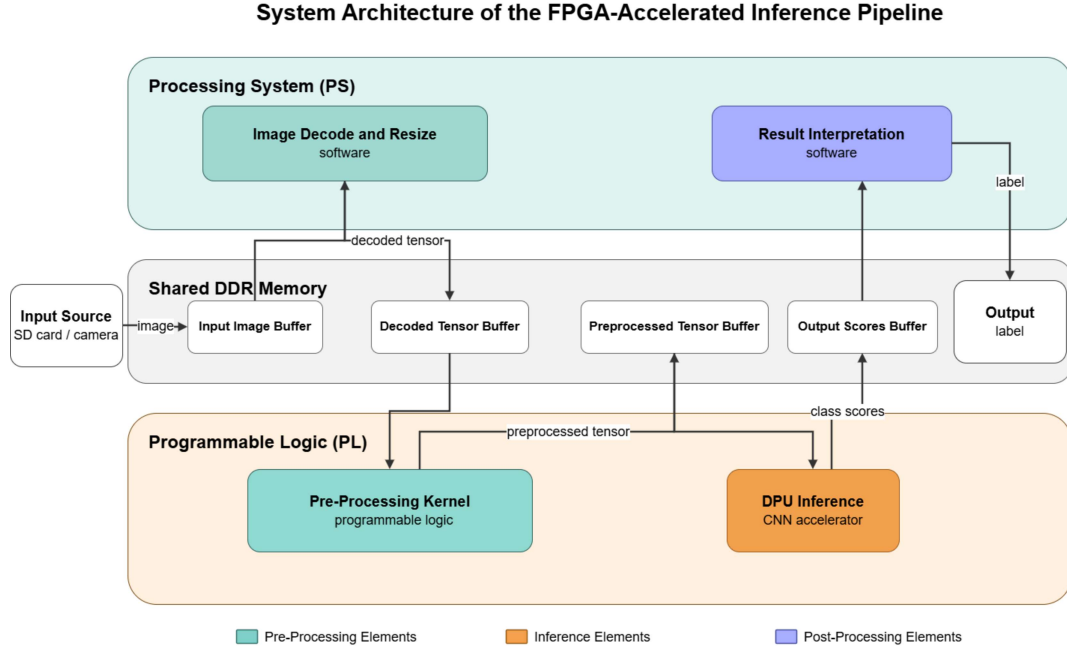


Figure 5: System Architecture for PL pre-processing applications

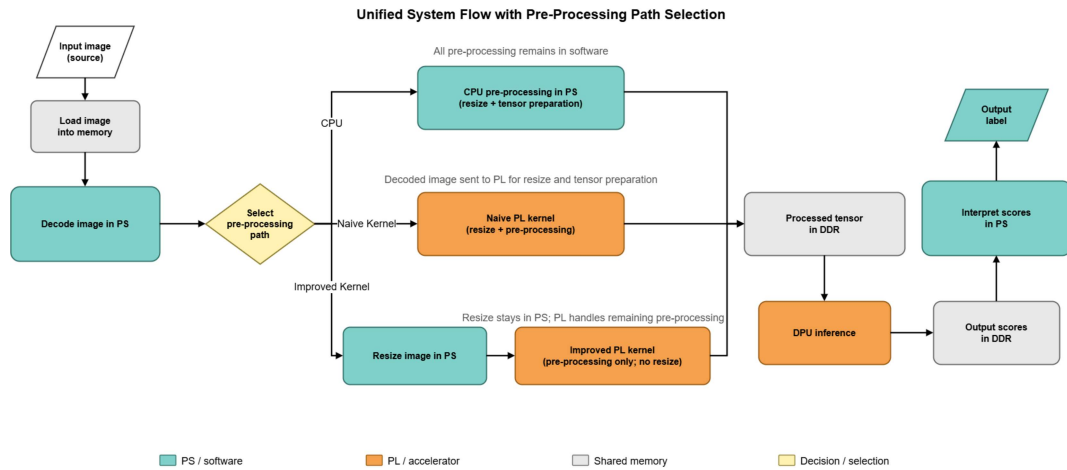


Figure 6: End-to-End Data Pipeline with Pre-Processing Path Selection

7 Implementation

7.1 Development Environment

7.1.1 Windows Subsystem for Linux (WSL)

Development was carried out within Windows Subsystem for Linux (WSL). WSL provided a Linux userspace on the host machine without requiring a separate native Linux installation. This was necessary because significant parts of the FPGA and Vitis AI toolchain used in this project were Linux native or completely unsupported on Windows. Therefore, WSL provided a compromise because it was a Linux compatible development environment that integrated with the host system and was suitable for container based workflows without needing another full system installation.

7.1.2 Docker and Vitis AI

The Vitis AI toolchain was used for quantisation, compilation, and deployment of CNN models onto the DPU. To improve reproducibility and reduce dependency conflicts, the development environment was containerised using Docker. This ensured that the required frameworks, runtime libraries, and auxiliary tools could be managed in a controlled software environment.

For model development, the TensorFlow 2 workflow was selected because it was well comparable with the available AMD examples and documentation. Since the host machine included an NVIDIA GTX 1080Ti, a GPU-enabled TensorFlow 2 environment was needed for practical training times. However, AMD does not provide a directly usable pre-built container for this workflow, and the official build scripts failed due to outdated dependencies. A compatible precompiled Docker image was therefore found instead, with only minor configuration changes that were required for the project's workflow.

7.1.3 Ultra96v2 Board Support Packages (BSPs)

Board support resources are required in order to generate a usable target platform for the Ultra96-V2 across the Vivado, Vitis, and PetaLinux flows. In principle, these support packages provide the board definitions, platform scripts, hardware export flow, and software configuration needed to create a working embedded Linux system for a specific target. In practice, however, the Ultra96-V2 could not be treated as a fully current, well-supported board. The platform is no longer actively maintained as

a mainstream target, and this created compatibility issues throughout the toolchain [39].

The board support flow used in this project was therefore based on vendor provided resources, particularly the Tria platform tree and the older Avnet/Ultra96-V2 support material on which that ecosystem ultimately depends. These resources provided the starting point for platform generation, including the base Ultra96-V2 platform, build scripts, PetaLinux generation, and packaging structure used to integrate the DPU overlays. Without this layer, the project would have required substantially more low-level platform bring-up work before any machine-learning could begin.

A key difficulty was that the board support resources were not fully aligned with the software stack required by the project and many dependencies were deprecated or discontinued. The final implementation used a Vitis AI 3.5 deployment flow together with a newer XRT/PetaLinux environment, and these components did not integrate cleanly in their stock form. In particular, VART 3.5 still expected a number of legacy XRT `xc1*` APIs, including functions such as `xc10pen`, `xc1Close`, `xc1ExecBuf`, and `xc1IPName2Index`, which were no longer link-exported in the XRT 2024.2 environment used in the project. As a result, the BSP layer could not simply be used as supplied. It had to be extended with custom compatibility patches and recipe modifications.

This patching work formed a large component of the implementation work. Custom `bbappend` layers were introduced for VART, XIR, Vitis AI Library, and XRT in order to restore build compatibility and preserve the expected DPU deployment flow. The VART patches reintroduced compatibility with the required legacy XRT interface. The XIR layer was adapted to build correctly against newer `protobuf` and `pybind11` configurations. The Vitis AI Library layer was modified to avoid build failures associated with newer `protobuf` and test dependencies. Finally, the XRT layer was extended to restore the Python and `pyxrt` functionality required by the benchmarking applications, and to provide an `xdputil` compatibility wrapper over `xbutil`. These changes were necessary to make the target usable as a complete platform after changes to dependency availability.

7.2 Hardware Platform

7.2.1 DPU Integration

All overlays used a single DPUCZDX8G instance with the same B1152 configuration, the same clocking scheme, and near-identical routed DPU resource usage. This is summarised in Table 4.

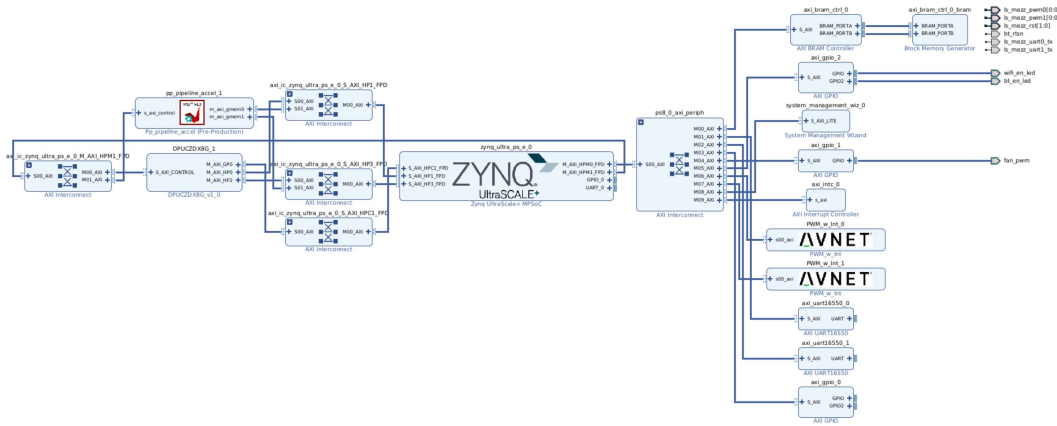


Figure 7: Interface View Block Diagram showing the DPU integrated within the system

Table 4: DPU configuration and routed DPU-only resource usage across overlays

Overlay	DPU Config	DPU Count	Clock(s)	LUT	REG	DSP
Dogs vs. Cats (naive)	B1152	1	300 / 600 MHz	32997	47615	222
Dogs vs. Cats (improved)	B1152	1	300 / 600 MHz	33010	47352	222
MNIST	B1152	1	300 / 600 MHz	32895	47733	222
ASL / ResNet50	B1152	1	300 / 600 MHz	33021	47466	222

7.2.2 Pre-Processing Kernel

The accelerator was implemented in High-Level Synthesis (HLS) instead of handwritten RTL. This kept the design easier to modify while still supporting AXI interfaces, loop pipelining, array partitioning, and packed data movement. Although custom RTL might potentially deliver higher performance, it would also increase development effort and reduce portability across different models and input formats.

Two different Dogs vs. Cats kernel variants were implemented. The first, the naive PL kernel, accepted arbitrary input dimensions and performed the full path from raw BGR pixels to DPU input tensor. This included resizing to 224×224 , BGR-to-RGB conversion, and int8 quantisation using the model scale factor. It showed that the complete pre-processing path could be offloaded into PL, but it also kept resize coordinate calculation and per-pixel quantisation arithmetic inside the hot loop. Combined with relatively narrow memory accesses, this limited the loop to an initiation interval of 8. Therefore, it was a correct proof of concept, but likely not the most efficient method of partitioning.

Listing 1: Naive Dogs-vs-Cats PL pre-processing kernel (simplified HLS pseudocode)

```

1 extern "C" void pp_pipeline_accel(
2     const uint8_t* in_bgr,
3     int8_t* out_rgb_q,
4     int in_w,

```

```

5     int in_h,
6     int fix_scale) {
7 #pragma HLS INTERFACE m_axi      port=in_bgr      bundle=gmem0
8 #pragma HLS INTERFACE m_axi      port=out_rgb_q    bundle=gmem1
9 #pragma HLS INTERFACE s_axilite  port=in_w         bundle=control
10 #pragma HLS INTERFACE s_axilite  port=in_h         bundle=control
11 #pragma HLS INTERFACE s_axilite  port=fix_scale    bundle=control
12 #pragma HLS INTERFACE s_axilite  port=return       bundle=control
13     const int OUT_W = 224;
14     const int OUT_H = 224;
15     for (int y = 0; y < OUT_H; y++) {
16         int src_y = (y * in_h) / OUT_H; // nearest-neighbour resize
17         for (int x = 0; x < OUT_W; x++) {
18 #pragma HLS PIPELINE II=8
19             int src_x = (x * in_w) / OUT_W;
20             int in_idx = (src_y * in_w + src_x) * 3;
21             uint8_t b = in_bgr[in_idx + 0];
22             uint8_t g = in_bgr[in_idx + 1];
23             uint8_t r = in_bgr[in_idx + 2];
24             // Quantise each channel using model scale
25             uint8_t rq = (r * fix_scale) / 255;
26             uint8_t gq = (g * fix_scale) / 255;
27             uint8_t bq = (b * fix_scale) / 255;
28             // Write output in RGB order
29             int out_idx = (y * OUT_W + x) * 3;
30             out_rgb_q[out_idx + 0] = (int8_t)rq;
31             out_rgb_q[out_idx + 1] = (int8_t)gq;
32             out_rgb_q[out_idx + 2] = (int8_t)bq;
33         }
34     }
35 }

```

The second, the improved PL kernel, changed the hardware-software boundary instead. It assumed that the input image had already been resized to 224×224 , so resize was removed from hardware and performed entirely in software when required. Inside the kernel, byte-wise transfers were replaced by 64-bit reads and packed 32-bit writes, and repeated multiply/divide quantisation was replaced by a precomputed lookup table. This reduced the arithmetic cost of the main pixel loop allowing for pipelining at initiation interval 1.

Listing 2: Improved Dogs-vs-Cats PL pre-processing kernel (simplified HLS pseudocode)

```

1 extern "C" void pp_pipeline_accel(
2     const ap_uint<64>* in_bgr,
3     uint32_t* out_rgb_q,
4     int in_w,
5     int in_h,
6     int fix_scale) {
7 #pragma HLS INTERFACE m_axi      port=in_bgr      bundle=gmem0 max_widen_bitwidth=64
8 #pragma HLS INTERFACE m_axi      port=out_rgb_q    bundle=gmem1 max_widen_bitwidth=32
9 #pragma HLS INTERFACE s_axilite  port=in_w         bundle=control
10 #pragma HLS INTERFACE s_axilite  port=in_h         bundle=control
11 #pragma HLS INTERFACE s_axilite  port=fix_scale    bundle=control
12 #pragma HLS INTERFACE s_axilite  port=return       bundle=control
13     const int W = 224;
14     const int H = 224;
15     const int PIXELS = W * H;
16     uint8_t scale_lut[256];
17 #pragma HLS ARRAY_PARTITION variable=scale_lut complete
18     // Precompute quantisation once

```

```

19     for (int i = 0; i < 256; i++) {
20 #pragma HLS PIPELINE II=1
21         scale_lut[i] = (i * fix_scale) / 255;}
22
23     ap_uint<64> word_lo = in_bgr[0];
24     ap_uint<64> word_hi = in_bgr[1];
25     int next_word = 2;
26     int byte_offset = 0;
27
28     for (int pix = 0; pix < PIXELS; pix++) {
29 #pragma HLS PIPELINE II=1
30         // Extract one BGR pixel from packed 64-bit stream
31         uint8_t b, g, r;
32         extract_bgr_from_words(word_lo, word_hi, byte_offset, b, g, r);
33         // Replace repeated arithmetic with LUT lookups
34         uint8_t bq = scale_lut[b];
35         uint8_t gq = scale_lut[g];
36         uint8_t rq = scale_lut[r];
37         // Pack quantised RGB into one 32-bit output word
38         out_rgb_q[pix] = (rq << 16) | (gq << 8) | bq;
39         // Advance packed input stream
40         byte_offset += 3;
41         if (byte_offset >= 8) {
42             byte_offset -= 8;
43             word_lo = word_hi;
44             word_hi = in_bgr[next_word++];
45         }
46     }
47 }

```

Table 5: Summary of pre-processing kernel implementation results

Kernel	Cycles	Time	LUT	REG	BRAM	DSP	WNS (ns)
Dogs vs. Cats (naive)	401,554	2.677 ms	2643	2851	1	20	0.038
Dogs vs. Cats (improved)	50,655	0.338 ms	4268	5358	1.5	6	0.018
MNIST	1261	8.407 μ s	3481	5246	1.5	6	0.009
ASL / ResNet50	50,654	0.338 ms	6407	9495	1.5	0	0.006

The improved kernel consumes more logic while using less arithmetic hardware. Removing repeated multiply/divide operations from the hot loop significantly reduced the DSP demand, while the introduction of packed word handling, lookup-table logic, and tighter scheduling expectedly increased LUT and FF usage. The improved design replaced arithmetic cost with dataflow and control logic, which was a favourable trade on this platform because the DPU itself already placed heavy demand on the board’s DSP resources.

Additional task-specific kernels were also implemented. The MNIST kernel used a fixed 28×28 grayscale path and packed the output into the format expected by the DPU, with a very small latency of 1261 cycles (8.407 μ s). The ASL/ResNet50 kernel also used a fixed 224×224 input, packed memory access, and signed int8 quantisation with per-channel mean subtraction, based on the improved kernel. Together, these kernels demonstrate that the general approach is not limited to a single benchmark, but can be adapted across multiple model families while remaining within a relatively

accessible HLS design style.

Overall, the pre-processing kernel should be considered as a reusable, model-adaptable hardware component. Its value lies not just in the performance improvements observed for workloads, but also in the fact that it offers a middle ground between pure software pre-processing and highly custom RTL-based hardware design. For developers using a DPU based flow, this might be useful. It enables selective hardware acceleration of front-end pipeline stages without requiring the entire system to be redesigned.

7.3 Software Platform

7.3.1 MobileNetV2 Dogs vs. Cats Model

The Dogs vs. Cats workflow follows the standard deployment path used throughout the project. The Kaggle dataset was converted into TFRecords with an 80:20 split per class, giving 20,000 training images and 5,000 test images in the preserved logs. A MobileNetV2-style network with $224 \times 224 \times 3$ input, inverted residual bottlenecks, global average pooling, dropout, and a two-class output layer was trained, pruned with `tf.nn.dct.IterativePruningRunner`, quantised, and compiled for the B1152 Ultra96 DPU configuration. The resulting development stages are summarised in Table 6.

Table 6: Dogs vs. Cats MobileNetV2 model-development stages

Stage	Accuracy	Parameters / Size
Trained	93.70%	2.26M params; 624.75M FLOPs $\sim$ 9MB
Pruned (ratio 0.2)	91.4%	1.43M params; 497.36M FLOPs $\sim$ 33MB
Transformed Slim	91.4%	<code>t_model.h5</code> $\sim$ 6.0 MB
Quantized	91.34%	<code>q_model.h5</code> $\sim$ 6.0 MB
Compiled	91.34%	<code>mobilenetv2.xmodel</code> $\sim$ 2.1 MB

7.3.2 MNIST Model

The MNIST workflow applies the same deployment structure to a grayscale digit-recognition task. The standard Keras MNIST split of 60,000 training images and 10,000 test images was used, with the test set also exported as grayscale JPEGs for target-side inference. The model is a small custom CNN with $28 \times 28 \times 1$ input, two convolution and max-pooling stages, a 128-unit dense layer, and a ten-class output layer. It was then pruned, quantised, and compiled for the same B1152 Ultra96 DPU target. The resulting model stages are summarised in Table 7.

Table 7: MNIST model-development stages

Stage	Accuracy	Parameters / Size
Trained	99.11%	421.6K params; 8.56M FLOPs $\sim$ 1.7 MB
Pruned (ratio 0.6)	99.12%	209.9K params; 3.76M FLOPs $\sim$ 6.6 MB
Transformed Slim	99.12%	<code>t_model.h5</code> $\sim$ 842 KB
Quantized	98.54 %	<code>q_model.h5</code> $\sim$ 150 KB
Compiled	98.54 %	<code>mnist.xmodel</code> $\sim$ 125 KB

7.3.3 ResNet50 ASL Model

The ASL workflow targets a heavier multi-class recognition task using the `ASL_Alphabet_Dataset` from Kaggle. The preserved configuration defines 29 output classes and forms the evaluation split by combining a class-wise hold-out from the training set with the official test images. The classifier is a ResNet50-based model with $224 \times 224 \times 3$ input, `include_top=False`, global average pooling, dropout, and a 29-class output layer, trained from scratch in the preserved configuration. Alongside the baseline model, a pruned variant was produced through iterative pruning, followed by transformation, quantisation, and compilation for the Ultra96 DPU. The baseline and pruned development stages are summarised in Tables 8 and 9. The pruned model was used for on-board benchmarking as it was $2.71\times$ faster in inference and had better accuracy.

Table 8: ASL baseline model-development stages

Stage	Accuracy	Parameters / Size
Trained	98.24%	23.7M params; 7.7G FLOPs $\sim$ 91 MB
Quantized	98.03%	<code>q_model.h5</code> $\sim$ 91 MB
Compiled	98.03%	<code>resnet50_asl.xmodel</code> $\sim$ 24 MB

Table 9: ASL pruned model-development stages

Stage	Accuracy	Parameters / Size
Pruned	99.71 %	8.17M params; 3.12G FLOPs $\sim$ 370 MB
Transformed Slim	99.78 %	<code>t_model.h5</code> $\sim$ 32 MB
Quantized (Pruned)	99.76 %	<code>q_model.h5</code> $\sim$ 32 MB
Compiled (Pruned)	99.76 %	<code>resnet50_asl.xmodel</code> $\sim$ 9.0 MB

7.3.4 Software Application

The target-side software application is derived from the multithreaded Vitis AI example flow and provides the runtime layer for end-to-end execution on the Ultra96-V2 [40]. It is responsible for loading the compiled model, preparing inputs, invoking

the DPU, and decoding outputs into predicted classes. Project-specific changes extend this baseline to support the Dogs vs. Cats, MNIST, and ASL workloads, together with both CPU-side and PL-side pre-processing paths. In the CPU path, image handling and tensor preparation are performed in software; in the PL path, raw image data is sent to the pre-processing kernel and returned as a DPU-ready tensor for inference.

7.3.5 PetaLinux Platform

PetaLinux was used to generate the deployable Linux images for the Ultra96-V2 and to package the runtime environment required by the DPU overlays and benchmark applications. The final platform includes the DPU software stack, Python and OpenCV support, and the firmware packages needed to load the project-specific overlays. Its role in this project was to provide a stable and repeatable software environment aligned with the implemented hardware configurations; the platform-maintenance details are covered earlier in the BSP discussion.

8 Experimental Methodology

8.1 Performance Metrics

A single metric is not sufficient to characterise this complex an embedded AI pipeline. Raw throughput alone can hide poor latency, while latency alone does not show whether improvements come at the cost of higher energy use. For this reason, both directly measured quantities and derived performance-efficiency metrics had to be used. The measured quantities represent the observable behaviour of the platform during execution, while the derived quantities normalise those measurements by workload size and therefore allow fairer comparison across runs of different lengths.

Time was measured because it captures both responsiveness and throughput. In the context of image classification, total runtime determines how long a complete workload takes, while stage-level runtime identifies whether pre-processing, inference, or post-processing forms the dominant bottleneck. Power was measured because embedded systems are constrained not only by raw performance but also by energy availability, thermal limits, and practical performance-per-watt. Image count was also recorded, as this is the natural workload variable of the experiments and allows time and energy to be normalised into per-image terms.

The metrics used in the evaluation are summarised in Table 10.

Table 10: Measured and derived performance metrics used in the evaluation

Metric	Type	Definition	Why it matters
Image count, N	Measured	Number of images processed in one benchmark run	Defines workload size and is required to normalise runtime and energy
Total runtime, t_{total}	Measured	End-to-end application runtime for a run	Captures overall responsiveness and practical completion time
Pre-processing runtime, t_{pre}	Measured	Time spent in image preparation before DPU execution	Identifies whether the front end of the pipeline is a bottleneck
Inference runtime, t_{infer}	Measured	Time spent executing the model on the DPU, CPU, or GPU path	Indicates cost of model execution itself
Post-processing runtime, t_{post}	Measured	Time spent interpreting model outputs where recorded	Useful for confirming that performance differences are not dominated by result handling
Average power, P_{avg}	Measured	Mean measured power over a stage or whole run	Captures platform electrical demand during execution
Total energy, E	Derived	$E = P_{\text{avg}} \times t$	Indicates absolute energy cost of completing a workload
Throughput / FPS	Derived	$\text{FPS} = N/t_{\text{total}}$	Captures processing rate and sustained throughput
Latency per image	Derived	$L = t_{\text{total}}/N$	Captures average time cost of a single image within a run
Energy per image	Derived	$E_{\text{img}} = E/N$	Indicates efficiency in a form directly comparable across workload sizes
Frames per joule	Derived	$\text{FPJ} = N/E$	Alternative efficiency measure showing useful work per unit energy
Pre-processing fraction	Derived	$t_{\text{pre}}/t_{\text{total}} \times 100$	Shows how dominant pre-processing is within the overall pipeline

8.2 Test Platform

The experiments were conducted across two distinct classes of platform. The first was a desktop host system used to provide CPU and GPU reference measurements. The second was the embedded Ultra96-V2 target used for the main FPGA/DPU evaluation. The desktop platform provides useful performance context and represents the type of compute environment often used during model development, whereas the Ultra96-V2 represents an embedded edge deployment target where power, area, and integration constraints are substantially tighter.

Different power-measurement methods were required on the different platforms. On the FPGA target, board power was measured using an external INA226-based shunt monitor connected over I²C. On the host system, CPU energy was measured using inbuilt sensors and GPU power using `nvidia-smi`. These methods are not identical, and therefore the power comparisons should be interpreted carefully. In particular, the FPGA figures represent measured power on the externally monitored whole-board rail, while the host CPU and GPU figures are device-specific software-accessible measurements. Because of this the desktop results are best treated as reference baselines rather than directly equivalent whole-system power measurements.

8.2.1 GPU/CPU

The desktop reference platform consisted of an AMD Ryzen 5 3600 CPU and an NVIDIA GTX 1080Ti GPU. This combination represents a realistic desktop development system rather than an embedded edge target. Its role in the evaluation was therefore not to act as a like-for-like competitor to the Ultra96-V2, but to provide context for how the tested workloads behave on widely available general-purpose hardware.

The CPU path is relevant because it represents the simplest baseline. No additional accelerator hardware is required, and all pre-processing and inference can be performed in software. This mirrors common workflows in which a model is first validated on a conventional workstation before being migrated to a dedicated accelerator platform.

The GPU path is similarly relevant because it reflects the dominant high-throughput solution for neural-network execution in mainstream machine-learning workflows. A GPU can often deliver much higher raw throughput than an embedded FPGA platform, but does so in a substantially different power, thermal, and deployment regime. The NVIDIA GTX 1080Ti was released in 2017 and represents an older high-end consumer GPU based on the `Pascal` architecture, from before the widespread in-

clusion of dedicated tensor compute units in consumer devices [41].

For the ASL host benchmarks, preprocessing was matched to the ResNet50 input requirements: images were loaded in BGR format, resized where required, cast to `float32`, and mean-subtracted prior to inference. GPU power was sampled periodically using `nvidia-smi --query-gpu=power.draw`. In both host cases, warm-up was performed before energy monitoring began, so the reported host energy values correspond to post-warm-up execution rather than one-time initialisation cost.

8.2.2 Ultra96-V2

The Ultra96-V2 is built around a Zynq UltraScale+ MPSoC that combines Arm Cortex-A53 processing cores with programmable logic on a single device. This makes it a suitable embedded target for heterogeneous computing, since Linux-based application control can remain on the processing system while throughput-oriented stages are implemented in hardware. In this work, that combination provides a realistic platform for examining full-pipeline CNN deployment under the resource and power constraints typical of embedded systems.

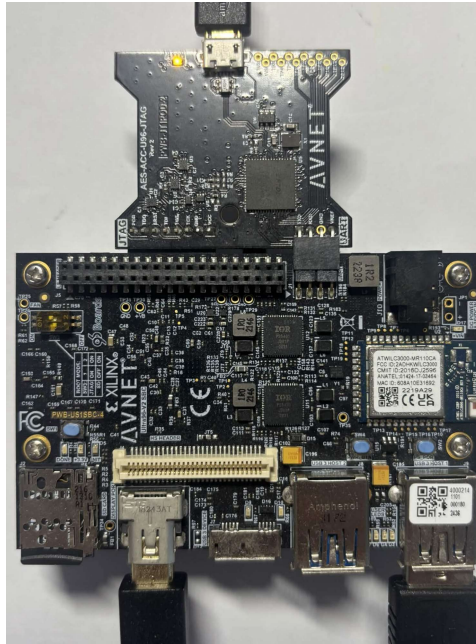


Figure 8: Experimental setup showing the Ultra96-V2 with USB webcam input, mini-DisplayPort output, and UART connection.

The board is also well suited to this evaluation because it supports AMD’s DPU-based Vitis AI deployment flow while still leaving programmable-logic capacity for application-specific hardware. This makes it possible to study pre-processing place-

ment around a fixed inference stage within one platform. The Ultra96-V2 is no longer a mainstream vendor target, and the software-maintenance work needed to preserve a usable tool flow has already been detailed in Section 7.1.3. It provides a compact embedded system on which inference acceleration, front-end offload, and whole-system power measurement can be examined together.

The age of the hardware platforms used in this work also allows an assessment of whether older or discontinued systems can still be reused for image-recognition tasks. Positive results would support the continued use of legacy hardware that might otherwise be discarded, which is particularly relevant under budget, supply, or other access constraints.

8.2.3 Ultra96-V2 Power Measurement Sub-Board

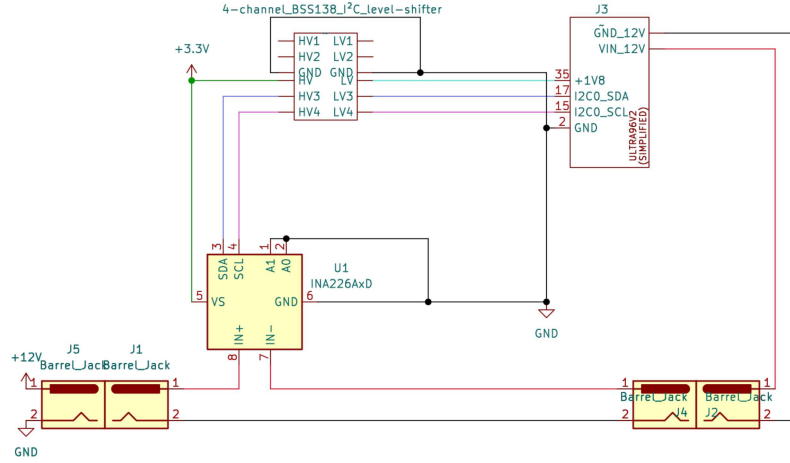


Figure 9: Schematic of Ultra96-V2 Power Measurement Sub-Board

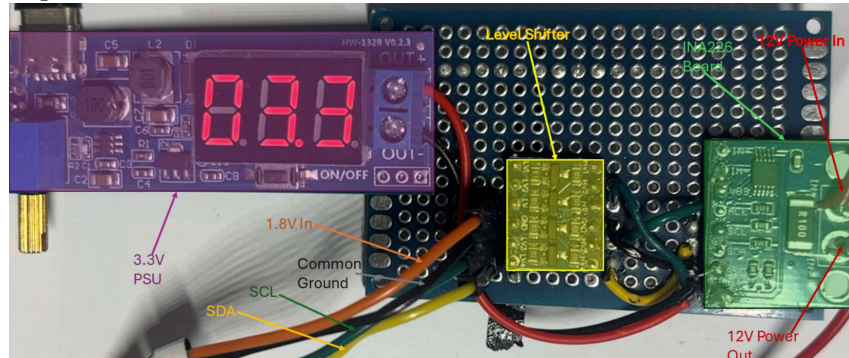


Figure 10: Annotated photograph of the implemented Ultra96-V2 power-measurement sub-board,

Power measurement on the Ultra96-V2 was carried out using a custom external measurement board built for this project. The board was designed around an INA226

current/voltage/power monitor and was placed in series with the Ultra96-V2 main power input [42]. The input power cable was routed through the INA226 measurement path, allowing the power drawn by the complete system to be recorded at the 12 V input rail. This provided whole-board measurements during operation, covering the processing system, programmable logic, memory, and peripheral activity together.

The monitor interfaced with the Ultra96-V2 over I²C, with a BSS138-based level shifter used between the sensing board and the Ultra96-V2 header. The INA226 itself was powered from 3.3 V, while its I²C signals were shifted down to 1.8 V for safe communication with the Ultra96-V2. On the software side, measurements were collected using a lightweight Python logger which accessed the INA226 directly through the Linux `i2c-dev` interface. The script configured the device for continuous conversion and sampled voltage and current periodically, from which power was obtained during benchmark execution. In the final methodology, these measurements were used to estimate average power and stage-level energy for the complete deployed pipeline.

8.3 Validation

Verification was used to confirm that the developed system was functionally correct, numerically consistent, and stable before performance and energy results were interpreted. This included checking model behaviour across floating-point, quantised, and deployed forms; confirming the correctness of CPU and PL pre-processing paths; and validating the power-measurement used in benchmarking.

Table 11: Summary of verification activities

Validation Target	Verification Method
Floating-point model accuracy	After training, each model was evaluated on a separate labelled test set to confirm that accuracy was acceptable.
Pruned and quantised model accuracy	The pruned and quantised models were evaluated on the same labelled test set to confirm that accuracy remained comparable.
Compiled model and DPU behaviour	The compiled model was executed on the test dataset to confirm that deployed accuracy remained comparable to the pruned and floating-point versions.
Kernel outputs	A small test script compared PL-kernel output tensors against CPU-generated tensors across the full test dataset to confirm consistency.
Repeated-run stability	Each image count was tested 10 times, with outliers identified and excluded where necessary.
Overlay selection	Overlay availability and loading were verified on-board using the <code>xmutil</code> tool.
Firmware loading	<code>xrt-smi examine --report dynamic-regions</code> was used on-board to inspect and confirm the loaded configuration.
INA226 readings	Readings were checked against a known load and voltage, with additional confirmation from multimeter measurements.

8.4 Testing Methodology

For the FPGA sweep experiments, image counts of 1, 5, 10, 25, 50, 75, 100, 150, 250, 500, 750, 1000, 1500, 2000, 2500, 3000, 4000, and 5000 were used, with 10 repeated runs per count. In this case, *count* refers to the number of images requested for processing in a single run and should not be interpreted as a DPU batch size. Images were enumerated sequentially from the target directory, and where the runtime internally packed work into runner buffers this occurred within the application itself. A single FPGA warm-up phase was executed before each set of runs, so all reported FPGA timing and power data correspond to the recorded benchmark runs directly.

Listing 3: Benchmark sweep and power logging harness (simplified pseudocode)

```
1 function run_one(app, count, rep, power_cfg):
```



```

2     start PowerLogger(power_cfg)
3     set stage = setup
4     launch worker subprocess
5     while subprocess is active:
6         read one output line
7         if output is a stage marker:
8             update logger stage
9         else if output is a result marker:
10            store returned application metrics
11        else:
12            write line to run log
13    set stage = teardown
14    stop PowerLogger
15    export raw power samples
16    compute stage statistics:
17        duration, average power, energy
18    merge power statistics with timing and accuracy results
19    return summary row
20
21 function benchmark_sweep(counts, reps):
22     for each count in counts:
23         for rep = 0 to reps-1:
24             row = run_one(app, count, rep, power_cfg)
25             append row to summary CSV

```

Stage-level timing was recorded using the benchmark and debug applications developed around the Vitis AI runtime. These applications measured application pre-processing time, inference time, and total time, with post-processing time recorded where relevant. The scripts emitted summary CSV files containing timing, power, energy, and accuracy.

On the FPGA target, power logging was performed using a dedicated **PowerLogger** thread built around the external INA226 monitor. The logger sampled periodically during execution and recorded stage-level average power values. Stage energy was then estimated as the product of average stage power and stage duration. No idle subtraction was applied, so the reported energy values correspond to gross energy on the monitored whole-board rail during each stage. On the host CPU and GPU baselines, timing and power were recorded after a warm-up phase to reduce the effect of one-time initialisation overheads.

9 Results

9.1 Dogs vs. Cats with MobileNetV2

The Dogs vs. Cats MobileNetV2 experiments provide the best test of performance, because pre-processing takes the majority of end-to-end execution time before hardware acceleration is applied.

9.1.1 No-Resize Results

Without resizing, the pre-processing pipeline is limited mostly to colour conversion, normalisation, and tensor preparation. This configuration is also relevant as in practice, fixed-size model-ready inputs are commonly used in AMD’s reference benchmarking flow for DPU-based CNN deployment [40].

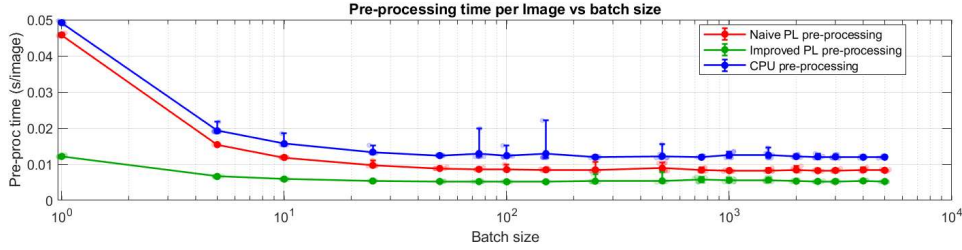


Figure 11: Pre-processing time per image for the no-resize Dogs vs. Cats MobileNetV2 pipeline.

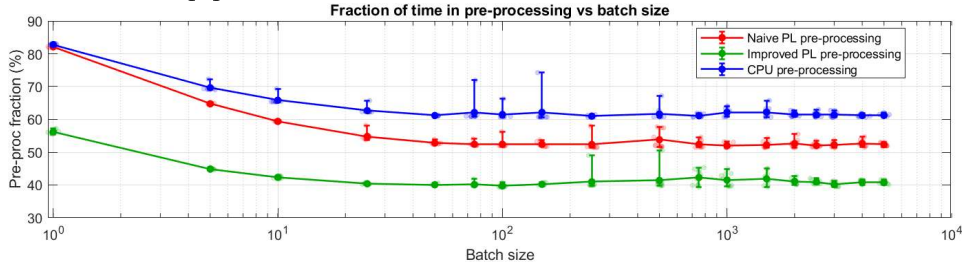


Figure 12: Fraction of total runtime spent in pre-processing for the no-resize Dogs vs. Cats MobileNetV2 pipeline.

Figure 11 shows that both hardware pre-processing variants outperform the CPU implementation across the full range of tested image counts. The naive kernel consistently reduces pre-processing time per image relative to the CPU, while the improved kernel provides the best results in all cases. The largest differences are observed at very small image counts, particularly for single-image runs, where the improved kernel is more than four times faster than the CPU implementation. As the image count increases, the measured time per image converges towards a stable value. This

stabilisation is reached by around 10 images for the improved kernel, whereas the CPU and naive-kernel results do not fully settle until around 50 images.

A similar pattern is visible in Figure 12, which shows the fraction of end-to-end runtime spent in pre-processing. The CPU path reaches a plateau at approximately 61%, the naive kernel at around 52%, and the improved kernel at around 41%. This is a significant result, because the improved kernel is the only configuration in which pre-processing no longer accounts for the majority of total runtime.

Each path has different amounts of variability. The CPU implementation shows the greatest variability, followed by the improved kernel, while the naive kernel is the most consistent. Despite this, even the worst-case results for the improved kernel remain better than the best-case results of the other two configurations, which reinforces the strength of the improvement.

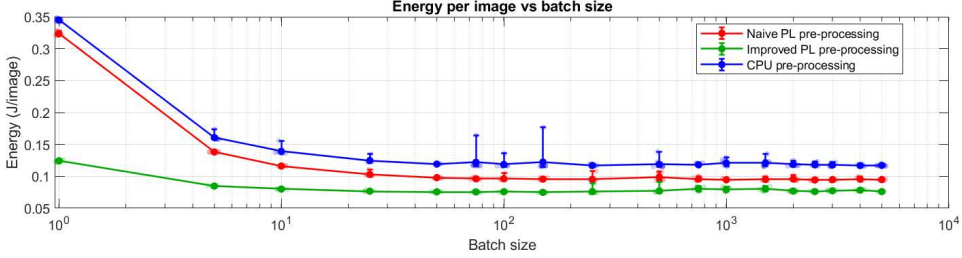


Figure 13: Energy per image for the no-resize Dogs vs. Cats MobileNetV2 pipeline.

The energy results in Figure 13 follow the same overall pattern as the timing results. This close correlation is expected, since the inference stage remains fixed across all three configurations and post-processing contributes relatively little to total execution cost. As a result, the main source of variation in energy per image is the amount of time spent in pre-processing. The improved kernel therefore achieves the best energy efficiency for the same reason that it achieves the lowest pre-processing time. This trend is seen for all Ultra96-V2 based tests.

9.1.2 Resize Results

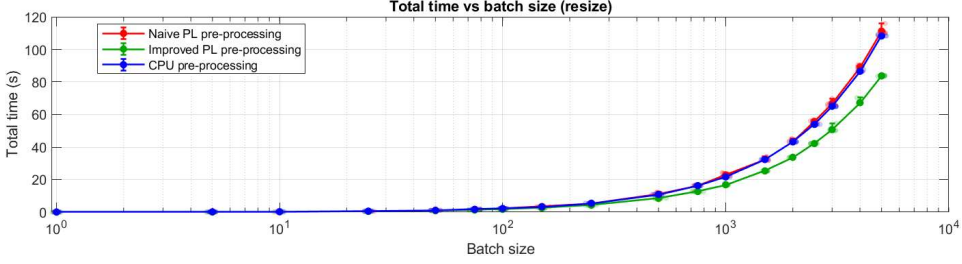


Figure 14: Total runtime for the resize-enabled Dogs vs. Cats MobileNetV2 pipeline.

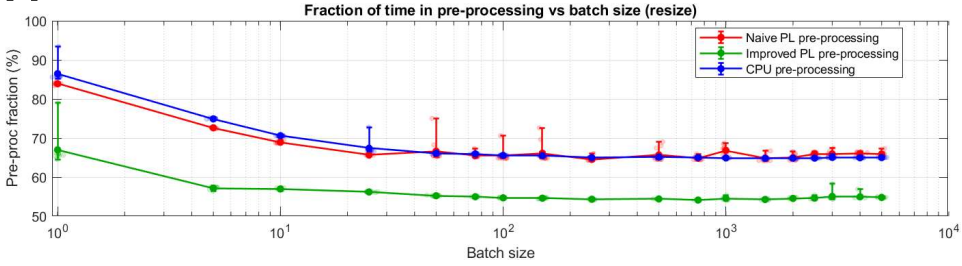


Figure 15: Fraction of total runtime spent in pre-processing for the resize-enabled Dogs vs. Cats MobileNetV2 pipeline.

The resize-enabled configuration is the more demanding and arguably the more realistic of the two Dogs vs. Cats pipelines, since raw input images will not always match the dimensions required by the classifier. Including resizing therefore increases the pre-processing workload substantially and provides a stronger test of whether hardware pre-processing remains beneficial when this additional cost is introduced.

Figure 14 shows that the resize-enabled case produces a different pattern from the no-resize results. The naive kernel and the CPU implementation are very similar across the full range of tested image counts, with the naive kernel becoming marginally worse at larger image counts. The improved kernel remains the best-performing configuration, but the gap is smaller than in the no-resize case, showing that the benefit of PL offload becomes more limited once the pipeline includes a heavier resize stage outside the optimised hardware path.

At larger image counts, where the results stabilise most clearly, the improved kernel requires only around 77% of the total runtime of the other two configurations. The effect is even more pronounced when considering pre-processing alone, where the improved kernel requires only around 64% of the time taken by the CPU and naive-kernel paths. This shows that the refined hardware-software partition still provides a meaningful gain, but that the gain is smaller than in the no-resize case,

The same effect is visible in Figure 15, which shows the fraction of end-to-end runtime spent in pre-processing. At larger image counts, the CPU path plateaus at approximately 65%, the naive kernel at around 66%, and the improved kernel at approximately 54%. Pre-processing therefore still dominates the pipeline in all three resize-enabled cases, but the improved kernel reduces that lead substantially. In contrast, the naive kernel does not improve the balance of the system and in practice behaves very similarly to the CPU implementation.

9.1.3 Discussion

In both the no-resize and resize-enabled cases, a large fraction of total runtime is spent before the model is even executed. This is also reflected in throughput. In real-world scenarios such as live camera inference, interactive vision systems, or high-volume image classification, this difference in sustained throughput may determine whether a system remains responsive or becomes bottlenecked before inference itself. In particular, 60 FPS is an important threshold because it matches the frame rate commonly used by cameras and displays, meaning that a pipeline operating at or above this rate is better positioned to support real-time image processing. Higher rates are similarly valuable in low-latency or high-speed scenarios, where the ability to process incoming frames faster than real-time provides additional headroom and reduces the risk of backlog.

The stage-level timing breakdown explains why the different pre-processing paths behave as they do. For the CPU implementation, image loading alone accounts for approximately 29.9% of total runtime, while the remaining software-side pre-processing stages are distributed across operations such as floating-point normalisation 13.5%, scaling 7.4%, and int8 conversion 3.9%. This produces a relatively heavy and fragmented front-end path in which a large amount of work is still performed on the processing system before inference begins.

The naive PL kernel improves on this, but only to a limited extent. Its pre-processing path is still dominated by image loading 28.7% of total runtime, and a further 21.1% of total runtime is spent in kernel wait time. This helps explain why the naive kernel does not produce a transformative improvement, particularly in the resize-enabled case. Resizing is not especially well suited to this style of simple PL kernel, because it introduces a more irregular and less efficient access and coordinate-mapping pattern than the rest of the pipeline. As a result, moving resize into hardware does not automatically lead to a better performance.

The improved kernel performs better because it changes both the workload and the style of computation. By moving resize back to the CPU, it leaves the PL kernel

with a more regular and better matched task: packed data conversion and quantisation on fixed-size input. This is reflected in the lower kernel-wait fraction, which falls to around 17.2% of total runtime, compared with 21.1% for the naive kernel. The improved path also makes more efficient use of memory movement. Instead of byte-wise reads and writes, it uses packed reads and writes, reducing overhead and allowing the hardware stage to process data in a more efficient streaming form. Similarly, repeated per-pixel quantisation calculations are replaced by lookup-table accesses, avoiding the need to perform the same arithmetic repeatedly within the hot loop. The unpack stage still contributes around 4.6% of total runtime, and kernel launch around 2.8%, but these costs are small relative to the gains achieved by simplifying and regularising the main body of the hardware work.

The naive kernel demonstrates that simply moving a computational stage into programmable logic does not guarantee better end-to-end behaviour. The improved kernel, by contrast, shows that a more careful partition of work between CPU and PL can materially reduce both runtime and energy per image.

9.2 MNIST with a Custom CNN

MNIST was included as a low-complexity contrast case rather than as a primary application benchmark. Unlike Dogs vs. Cats and ASL, MNIST consists of small 28×28 grayscale digit images with minimal background complexity and very limited pre-processing requirements.

9.2.1 End-to-End Results

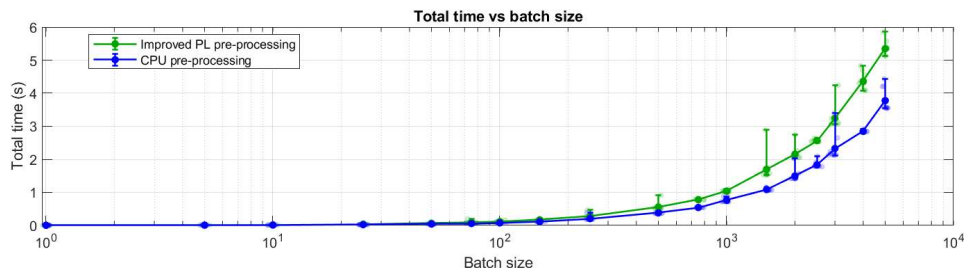


Figure 16: Total runtime for the MNIST pipeline.

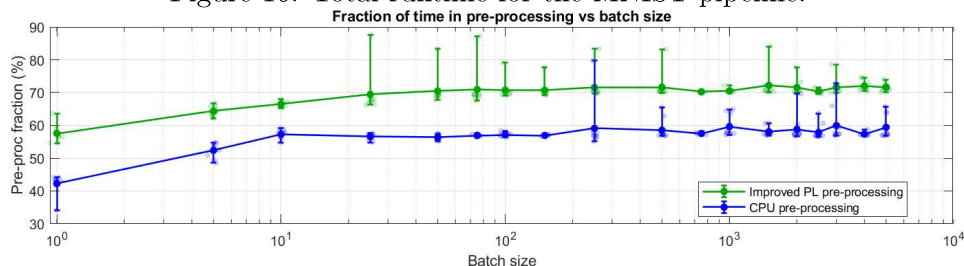


Figure 17: Fraction of total runtime spent in pre-processing for the MNIST pipeline.

The MNIST results show a very different pattern from the Dogs vs. Cats pipeline. Whereas PL pre-processing was beneficial for the larger RGB workload, the improved PL path performs worse than CPU pre-processing for MNIST in end-to-end terms. Figure 16 shows that total runtime remains lower for the CPU implementation across the full range of tested image counts, with the gap becoming more pronounced at larger workloads.

This difference is also visible in the fraction of time spent in pre-processing, shown in Figure 17. The improved PL path consistently spends a larger proportion of total runtime in pre-processing than the CPU path. At larger image counts, pre-processing accounts for roughly 70% of total runtime in the PL case, compared with around 55–60% for the CPU implementation. This is the opposite of the trend seen in Dogs vs. Cats, where moving pre-processing into programmable logic reduced the dominance of the front end of the pipeline.

9.2.2 Discussion

The reason for the behaviour seen here is that MNIST offers relatively little useful work for the hardware stage to accelerate. The inputs are already very small, grayscale rather than RGB, and require far less transformation than a natural-image classification task. As a result, the benefits of hardware execution do not outweigh

the overheads associated with moving data into and out of the PL pre-processing path. In a workload of this size, those overheads become a proportionally larger part of total execution time.

9.3 ASL with ResNet50

The ASL workload was included as a more application oriented evaluation case than MNIST and a heavier model case than Dogs vs. Cats. ResNet50 was also selected because it provides a substantially more demanding model than MobileNetV2.

9.3.1 CPU and GPU Results

The host CPU and GPU results provide a useful reference point for the ASL workload. They are not directly equivalent embedded comparisons to the Ultra96-V2, but they show how the same task behaves on more conventional general-purpose and high-throughput hardware and allow for an energy use comparison. This helps place the FPGA results within a wider practical context.

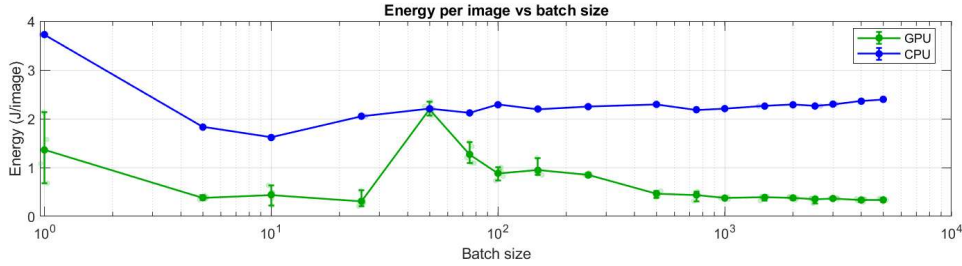


Figure 18: Energy per image for the ASL workload on host CPU and GPU.

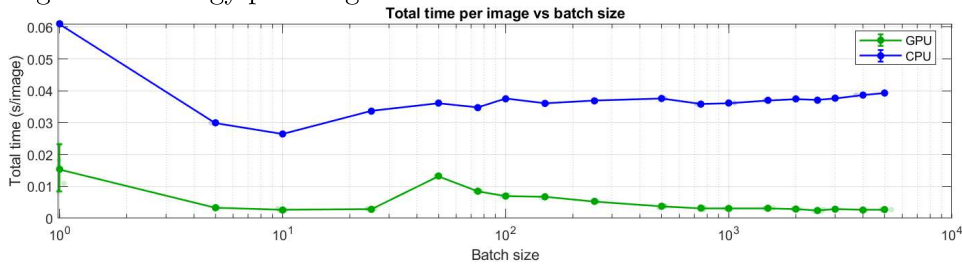


Figure 19: Total time per image for the ASL workload on host CPU and GPU.

Figures 18 and 19 show that the GPU substantially outperforms the CPU in both runtime and energy efficiency. At larger tested image counts, GPU time per image stabilises at only a few milliseconds, whereas the CPU remains around an order of magnitude slower. The same trend is visible in energy per image, where the GPU consistently requires far less energy than the CPU. This is the expected result

for a model such as ResNet50, whose size and arithmetic intensity are much better matched to a high-throughput GPU than to a conventional CPU execution path.

A local anomaly is visible in the GPU measurements at intermediate image counts. The GPU timing and energy per image shows a clear spike at 50 images, and the values at 75 and 100 images also remain elevated before returning to the main trend. This does not alter the overall conclusion that the GPU outperforms the CPU.

9.3.2 FPGA End-to-End Results

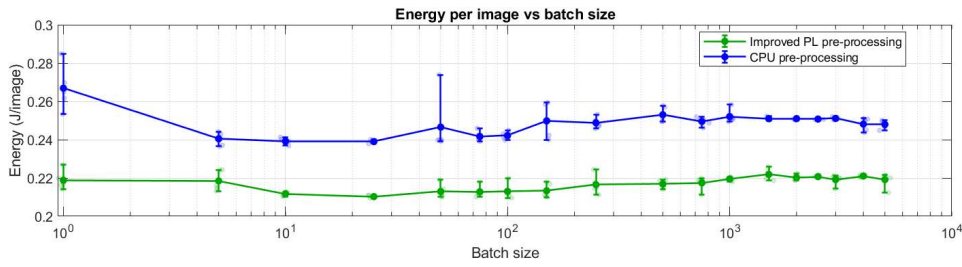


Figure 20: Energy per Image for ASL workload on FPGA

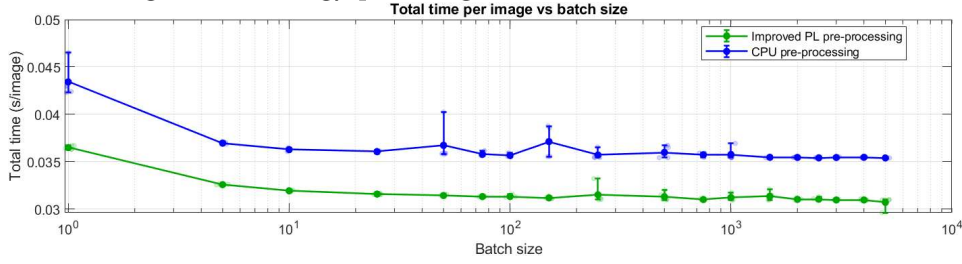


Figure 21: Time spent per Image for ASL workload on FPGA

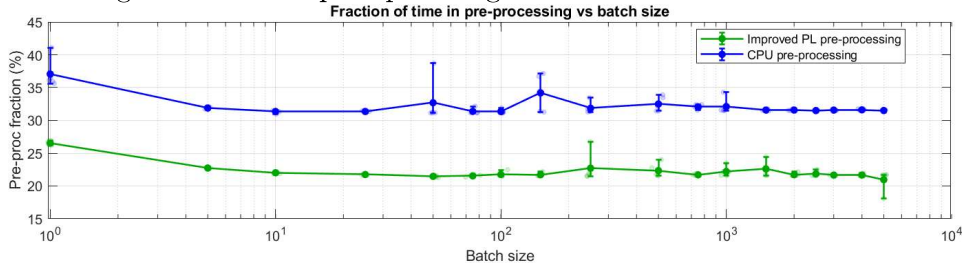


Figure 22: Fraction of total runtime spent in pre-processing for the ASL pipeline

Figures 20, 21, and 22 show that the improved PL path consistently outperforms CPU pre-processing for the ASL workload, even though the model itself is substantially heavier than in the Dogs vs. Cats case.

At the largest tested image count of 5000, CPU pre-processing required approx-

imately 55.81 s in total, compared with 31.88 s for the improved PL path. The total runtime fell from approximately 177.01 s to 153.14 s, corresponding to a reduction of about 13.5%. Energy per image fell from approximately 0.2477 J/image to 0.2189 J/image, a reduction of about 11.6%.

The fraction of runtime spent in pre-processing also decreases substantially. At larger image counts, the CPU path spends roughly one third of total runtime in pre-processing, while the improved PL path reduces this to around one fifth to one quarter. This is a smaller fraction than in Dogs vs. Cats, which reflects the heavier inference cost of ResNet50, but it is still large enough to influence end-to-end performance.

This is also reflected in throughput. At 5000 images, the CPU pre-processing path achieves approximately 28.2 FPS, while the improved PL path reaches roughly 32.6 FPS. The absolute improvement is smaller than in the Dogs vs. Cats case, but it remains meaningful for a workload of this size. Exceeding 30 FPS is a significant threshold because it corresponds to the frame rate commonly used by many cameras and real-time video systems [43].

9.3.3 Live Image Processing

To complement the benchmark results, the ASL pipeline was also deployed in a simple live image-processing setting using webcam input. This was intended as a practical proof of concept and not as a highly optimised production system. The aim was to show that the deployed model and pre-processing pipeline could operate on a continuous incoming image stream and produce classifications in real time.

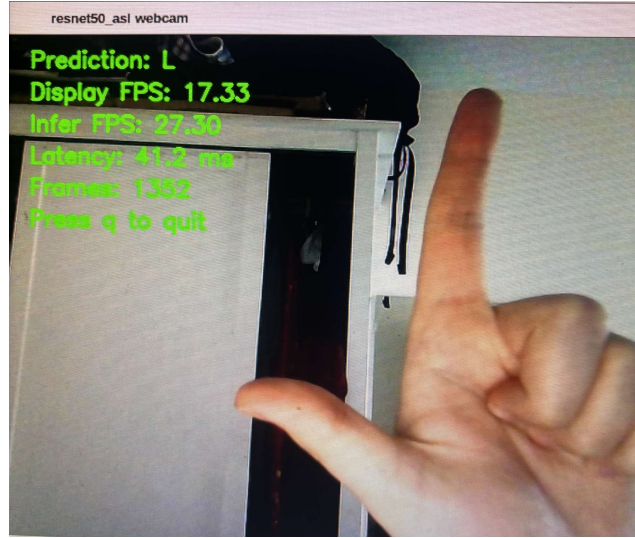


Figure 23: Example of the live ASL image-processing pipeline operating on webcam input.

Figure 23 shows an example of the live pipeline in use, implementing the pre-processing kernel. In this case, the system correctly classifies the displayed hand sign as L, while also reporting live performance statistics on screen. These values indicate that the system is able to operate interactively, even with a relatively crude webcam-based input setup. The gap between inference FPS and display FPS is expected, as the live pipeline includes additional overheads beyond model execution alone, including frame capture, display refresh, and general application control.

Although this live setup is relatively simple, it demonstrates that the platform is not limited to dataset processing. The same hardware-software pipeline can be applied to continuous real-world input, producing useful classifications in a form that is directly visible to the user.

9.3.4 Discussion

At the largest evaluated workload of 5000 images, the GPU remained the strongest platform in terms of raw latency, but the improved FPGA pipeline was both faster than the host CPU and more energy-efficient than either host reference, while also improving substantially on the FPGA’s own CPU-preprocessing baseline.

Within the FPGA platform, for 5000 images, moving from CPU pre-processing to the improved PL implementation increased throughput from 28.2 FPS to 32.6 FPS, pushing the system beyond the 30 FPS rate commonly associated with real-time camera operation [43]. At the same time, the results suggest that further gains could be acquired by reducing inference cost in this scenario, for example through

further model pruning, as this configuration is still highly inference dominated.

This interpretation is consistent with the live ASL demonstration, in which webcam-based inference was sustained at approximately 27.3 FPS with an end-to-end latency of about 41.2 ms.

9.4 Overall Discussion

The benefit of PL offload depended strongly on workload structure. Dogs vs. Cats and ASL retained enough regular front-end work for hardware pre-processing to improve overall behaviour, whereas MNIST did not. For the smaller grayscale workload, transfer and invocation overheads formed too large a share of the total cost for the PL path to repay itself. Pre-processing offload was therefore most useful when the remaining front end was both substantial and well suited to a regular hardware data path.

The results also show that implementation strategy mattered as much as placement. The strongest gains did not come from moving the largest possible amount of work into programmable logic, but from choosing a boundary that matched the platform well. The improved kernel performed better because it operated on a fixed-size path, used packed transfers, and simplified inner-loop arithmetic. The naive version was less effective because it combined offload with a resize-heavy path that was poorly matched to this form of hardware. Pre-processing acceleration was a partitioning problem. HLS proved useful here because it provided a practical way to accelerate model-specific front-end stages across multiple workloads. More generally, the results support a more selective view of FPGA acceleration around reusable overlays. The DPU provides a stable inference core, but worthwhile gains still depend on careful judgement about which surrounding stages should remain in software and which justify dedicated hardware.

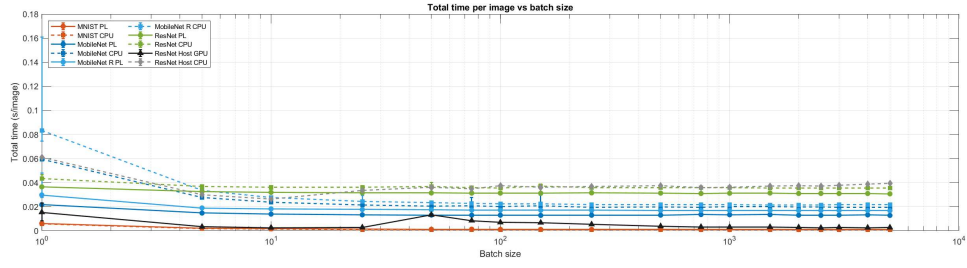


Figure 24: Summary comparison of per-image runtime versus image count for MNIST, MobileNetV2, and ResNet50

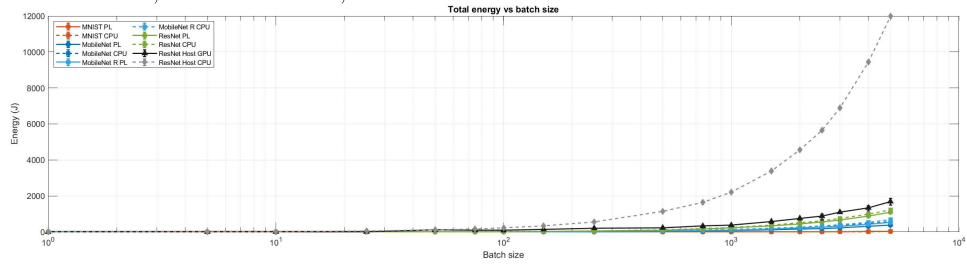


Figure 25: Summary comparison of total energy versus image count for MNIST, MobileNetV2, and ResNet50

10 Conclusions

This project examined how pre-processing placement affects end-to-end CNN deployment on an embedded FPGA platform when inference is already executed on AMD’s DPU. Using the Ultra96-V2, it compared a fixed DPU inference path with CPU and programmable-logic pre-processing across three workloads: Dogs vs. Cats with MobileNetV2, MNIST with a custom CNN, and ASL recognition with ResNet50. The completed system included reusable HLS kernels, automated benchmarking, whole-board power measurement, and a repaired software stack that kept a discontinued platform usable within a modern deployment flow. Therefore, it contributed both a controlled evaluation of embedded CNN deployment and a practical demonstration that ageing hardware can still support a usable modern deployment path.

The results show that pre-processing can remain a significant part of total runtime and energy even after inference has been accelerated. For Dogs vs. Cats and ASL, the improved PL path increased throughput and reduced energy per image. In the stronger Dogs vs. Cats configuration, the pipeline exceeded 60 FPS. In ASL, it exceeded 30 FPS and made live operation more plausible on the embedded target. For MNIST, the inputs were too small and simple for the PL path to overcome its own overheads, so CPU pre-processing remained preferable.

The clearest design result is the contrast between the naive and improved kernels. Moving more of the front end into programmable logic was not enough on its own. The improved kernel performed better because it used a hardware–software boundary that better suited the platform, with fixed-size input, packed transfers, and simpler per-pixel processing. End-to-end behaviour was therefore dependent on how the surrounding data path was partitioned, not only on accelerating inference itself.

The study was limited to one board, one DPU configuration, and a small set of workloads. The desktop CPU and GPU comparisons were useful for context, but their power measurements were not directly comparable to the whole-board FPGA measurements. The live ASL demonstration also remained a proof of concept with visible software and display overheads. Even with these limits, the project showed that a discontinued platform could still support a usable modern CNN deployment path once the supporting software stack and evaluation flow had been repaired. Overall, the work shows that a reusable DPU overlay provides a viable inference core, but end-to-end performance still depends heavily on how data is prepared and delivered to it.

10.1 Objective-to-Evidence Summary

Table 12 maps the project objectives to the sections in which they were addressed, making it clearer where each objective was achieved and to what extent.

Table 12: Objective-to-evidence map

Priority	Objective	Evidence in report	Status
Must	Environment and deployment flow	Sections 7.1, 7.3.5	Fully achieved
Must	Baseline CNNs trained and deployed	Sections 7.3.1–7.3.3	Fully achieved
Must	Quantisation, compilation, and DPU deployment	Sections 7.3.1–7.3.5	Fully achieved
Must	Functional correctness and stability	Section 8.3	Fully achieved
Must	Throughput, latency, power, energy evaluation	Sections 8.1–8.4, 9	Fully achieved
Must	FPGA/CPU/GPU comparison	Sections 8.2.1, 9.3.1–9.3.4	Fully achieved
Should	Hardware pre-processing pipeline	Section 7.2.2; Sections 9.1–9.3	Fully achieved
Should	Model optimisation trade-offs	Sections 7.3.1–7.3.3	Mostly achieved
Should	Automated benchmarking and reproducibility	Sections 7.1.2, 7.3.4–7.3.5, 8.4	Fully achieved
Could	Real-time image processing	Section 9.3.3	Partially achieved
Could	More complex CNN evaluation	Sections 7.3.3, 9.3	Partially achieved
Must	Usefulness of FPGA for edge AI	Sections 9.4, 10	Fully achieved

11 Further Work

A clear extension of this work is to widen the workload set used to test pre-processing placement. The current results already show that workload structure determines

whether programmable-logic offload is worthwhile, but that conclusion would be stronger across a wider range of model families and data paths. Future work should therefore include models with heavier normalisation, per-channel scaling, multi-branch input formatting, more demanding output handling, and significant post-processing. This would extend the study beyond front-end preparation towards the question of how much of the full application path around a reusable inference core should be treated as accelerator design [7][8][38].

Another direction would be to improve the pre-processing hardware itself. The current improved kernel already shows that careful partitioning can produce worthwhile gains, but it still leaves inefficiencies between software and the DPU. One direct refinement would be to write a DPU-ready tensor layout from the kernel, removing the separate unpack stage on the processing system. Another would be to move earlier input stages into hardware, especially JPEG decode, to test whether some of the image-loading cost can also be reduced in programmable logic [7]. The resize stage also deserves closer study. The poor result of the resize-heavy kernel here should not be taken as a general result against hardware resizing. A strong follow-up around building a resize unit designed for streaming access, predictable buffering, and packed data movement, then comparing it against the current CPU-plus-PL split, would be beneficial. That stage could also be compared with Vitis Vision implementations and handwritten HDL to judge how much performance the current HLS approach leaves unused.

A third extension could examine stage interaction more closely. The present study compares CPU and PL responsibilities mainly as separate configurations, which is useful for control, but still leaves idle time between stages. A deeper systems study would pipeline pre-processing, inference, and output handling more aggressively so that stages overlap where possible. A related step would be to investigate direct camera or near-sensor input paths. The current live ASL demonstration still depends on a standard webcam-based software flow, so there remains scope to move image acquisition closer to the hardware pipeline and connect the work more directly to the wider embedded-vision discussion of visual computation across the continuum from near-sensor to in-sensor processing [26][38].

Finally, the evaluation could be widened across other boards and other DPU configurations. This would show whether the same partitioning lessons are true when the balance between PS, PL, and inference changes. Comparison with a wider range of host systems would also give more CPU and GPU context.

12 Bibliography

References

- [1] R. Singh and S. S. Gill, “Edge AI: A survey,” *Internet of Things and Cyber-Physical Systems*, vol. 3, 03 2023.
- [2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, vol. 3, pp. 637–646, 10 2016.
- [3] M. Satyanarayanan, “The emergence of edge computing,” *Computer*, vol. 50, pp. 30–39, 01 2017.
- [4] F. Yan, A. Koch, and O. Sinnen, “A survey on FPGA-based accelerator for ML models.” [Online]. Available: <https://arxiv.org/html/2412.15666v1>, 2024.
- [5] E. Nurvitadhi, G. Venkatesh, J. Sim, D. Marr, R. Huang, J. Ong Gee Hock, Y. T. Liew, K. Srivatsan, D. Moss, S. Subhaschandra, and G. Boudoukh, “Can FPGAs beat GPUs in accelerating next-generation deep neural networks?,” *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pp. 5–14, 02 2017.
- [6] AMD, “DPU for convolutional neural network.” [Online]. Available: <https://www.amd.com/en/products/adaptive-socs-and-fpgas/intellectual-property/dpu.html>, 03 2025.
- [7] T.-Y. Li, F. Zhang, W. Guo, J.-L. Shen, and M.-Q. Sun, “An FPGA-based JPEG preprocessing accelerator for image classification,” *The Journal of Engineering*, vol. 2022, pp. 919–927, 07 2022.
- [8] E. Cittadini, M. Marinoni, and G. Buttazzo, “A hardware accelerator to support deep learning processor units in real-time image processing,” *Engineering Applications of Artificial Intelligence*, vol. 145, p. 110159, 02 2025.
- [9] Avnet, *Ultra96-V2 Single Board Computer Hardware User’s Guide*. Avnet, 2021.
- [10] Y. Le Cun, L. Jackel, B. Boser, J. Denker, H. Graf, I. Guyon, D. Henderson, R. Howard, and W. Hubbard, “Handwritten digit recognition: applications of neural network chips and automatic learning,” *IEEE Communications Magazine*, vol. 27, pp. 41–46, 11 1989.

- [11] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet classification with deep convolutional neural networks,” *Communications of the ACM*, vol. 60, pp. 84–90, 05 2012.
- [12] P. Kumar, Pooja, N. Chauhan, and N. Chaurasia, “A vision-based pothole detection using CNN model,” *SN Computer Science*, vol. 4, 09 2023.
- [13] A. Shawahna, S. M. Sait, and A. El-Maleh, “FPGA-based accelerators of deep learning networks for learning and classification: A review,” *IEEE Access*, vol. 7, pp. 7823–7859, 2018.
- [14] Kaggle, “Dogs vs. cats.” [Online]. Available: <https://www.kaggle.com/competitions/dogs-vs-cats>.
- [15] Y. Lecun, “MNIST handwritten digit database, yann lecun, corinna cortes and chris burges.” [Online]. Available: <https://yann.lecun.com/exdb/mnist/>.
- [16] Kaggle, “ASL alphabet.” [Online]. Available: <https://www.kaggle.com/dsv/29550>.
- [17] AMD, “Zynq UltraScale+ MPSoC data sheet: Overview.” [Online]. Available: <https://docs.amd.com/api/khub/documents/sbPbXcMUiRSJ205STvuGNQ/content>.
- [18] Xilinx, “Zynq UltraScale+ MPSoC embedded design methodology guide.” [Online]. Available: <https://docs.amd.com/v/u/en-US/ug1228-ultrafast-embedded-design-methodology-guide>, 04 2017.
- [19] AMD, “AMD Zynq™ UltraScale+™ MPSoCs.” [Online]. Available: <https://www.amd.com/en/products/adaptive-socs-and-fpgas/soc/zynq-ultrascale-plus-mpsoc.html>, 08 2024.
- [20] AMD, “DPU IP details and system integration — Vitis™ AI 3.5 documentation.” [Online]. Available: <https://xilinx.github.io/Vitis-AI/3.5/html/docs/workflow-system-integration.html>, 2022.
- [21] AMD, “DPUCZDX8G for Zynq UltraScale+ MPSoCs product guide (PG338).” [Online]. Available: <https://docs.amd.com/r/en-US/pg338-dpu/>, 01 2023.
- [22] AMD, “Vitis AI user guide (UG1414).” [Online]. Available: <https://docs.amd.com/r/1.4-English/ug1414-vitis-ai>, 07 2021.
- [23] Y. Fu, E. Yu, and A. Sirasao, “8-bit dot-product acceleration.” [Online]. Available: <https://docs.amd.com/api/khub/documents/Wr7TqDp2Zv68IugmRp9hwg/content>, 27 2017.

- [24] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, “Efficient processing of deep neural networks: A tutorial and survey,” *Proceedings of the IEEE*, vol. 105, pp. 2295–2329, 12 2017.
- [25] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE Journal of Solid-State Circuits*, vol. 52, pp. 127–138, 01 2017.
- [26] W. Fabre, K. Haroun, V. Lorrain, M. Lepecq, and G. Sicard, “From near-sensor to in-sensor: A state-of-the-art review of embedded AI vision systems,” *Sensors*, vol. 24, pp. 5446–5446, 08 2024.
- [27] Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers, “FINN,” *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pp. 67–74, 02 2017.
- [28] M. Blott, T. B. Preußer, N. J. Fraser, G. Gambardella, K. O’Brien, Y. Umuroglu, M. Leeser, and K. Vissers, “FINN-R,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 11, pp. 1–23, 12 2018.
- [29] S. I. Venieris and C.-S. Bouganis, “fpgaConvNet: Mapping regular and irregular convolutional neural networks on FPGAs,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 30, pp. 326–342, 02 2019.
- [30] X. Zhang, J. Wang, C. Zhu, Y. Lin, J. Xiong, W.-m. Hwu, and D. Chen, “DNNBuilder: an automated tool for building high-performance DNN hardware accelerators for FPGAs.” [Online]. Available: <https://ieeexplore.ieee.org/document/8587697>, 11 2018.
- [31] F. Fahim, B. Hawks, C. Herwig, J. Hirschauer, S. Jindariani, N. Tran, L. P. Carloni, G. Di Guglielmo, P. Harris, J. Krupa, D. Rankin, M. B. Valentin, J. Hester, Y. Luo, J. Mamish, S. Orgrency-Memik, T. Aarrestad, H. Javed, V. Loncar, M. Pierini, A. A. Pol, S. Summers, J. Duarte, S. Hauck, S.-C. Hsu, J. Ngadiuba, M. Liu, D. Hoang, E. Kreinar, and Z. Wu, “hls4ml: An open-source codesign workflow to empower scientific low-power machine learning devices.” [Online]. Available: <https://arxiv.org/abs/2103.05579>, 03 2021.
- [32] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, “TVM: An automated end-to-end optimizing compiler for deep learning,” 02 2018.
- [33] M. Machura, M. Danilowicz, and T. Kryjak, “Embedded object detection with custom littlenet, FINN and Vitis AI DCNN accelerators,” *Journal of Low Power Electronics and Applications*, vol. 12, pp. 30–30, 05 2022.

- [34] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2018.
- [35] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, “Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1.” [Online]. Available: <https://arxiv.org/abs/1602.02830>, 03 2016.
- [36] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 770–778, 06 2016.
- [37] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 06 2018.
- [38] D. Kang, A. Mathur, T. Veeramacheneni, P. Bailis, and M. Zaharia, “Jointly optimizing preprocessing and inference for DNN-based visual analytics,” *arXiv (Cornell University)*, 01 2020.
- [39] TRIA, “TRIA Americas Products Manufacturing Update: End-of-life notification EOL25002.” [Online]. Available: <https://www.avnet.com/wcm/connect/f187b068-3671-44d0-8390-b4cc01ee2ba2/EOL25002+AES-ULTRA96-V2.pdf?MOD=AJPERES&ContentCache=NONE&CACHE=NONE>, 02 2025.
- [40] Xilinx, “Vitis-AI-Tutorials/Tutorials/TF2-Vitis-AI-Optimizer at 3.5 · Xilinx/Vitis-AI-Tutorials.” [Online]. Available: <https://github.com/Xilinx/Vitis-AI-Tutorials/tree/3.5/Tutorials/TF2-Vitis-AI-Optimizer/>, 06 2023.
- [41] NVIDIA, “The ultimate geforce GPU comparison.” [Online]. Available: <https://www.nvidia.com/en-us/geforce/graphics-cards/compare/>.
- [42] “INA226 36v, 16-bit, ultra-precise I2C output current, voltage, and power monitor with alert.” [Online]. Available: <https://www.ti.com/lit/ds/symlink/ina226.pdf>, 06 2011.
- [43] Sony, “What is the difference between 24p, 25p, 30p, 50p, 50i, 60p, and 60i video frame rates? — sony USA.” [Online]. Available: <https://www.sony.com/electronics/support/articles/00014471>, 01 2024.